# Pesquisa em Memória Primária

Prof. Henrique Batista da Silva

# Pesquisa em Memória Primária

- Como recuperar informação a partir de uma grande massa de informação previamente armazenada?
- A informação é dividida em registros:
  - Cada registro possui uma chave de pesquisa.
- O objetivo:
  - encontrar ocorrências de registros com chaves iguais à chave da pesquisa.
  - A pesquisa pode ser com sucesso ou sem sucesso

# Pesquisa em Memória Primária

- Há diversos métodos de pesquisa;
- A escolha do método mais apropriado, depende:
  - Quantidade dos dados envolvidos;
  - Arquivo estar sujeito a inserções e retiradas frequentes (o objetivo é minimizar o tempo de pesquisa, sem preocupação com o tempo de estruturar o arquivo).

# TADs de Pesquisa

- Importante considerar os algoritmos de pesquisa como **tipos abstratos de dados:**
  - um conjunto de operações associado a uma estrutura de dados
- Deve haver uma independência de implementação para as operações.
- Operações mais comuns
  - Inicializar a estrutura de dados.
  - Pesquisar um ou mais registros com determinada chave.

# TADs de Pesquisa

- Operações mais comuns (cont...)
  - Inserir um novo registro.
  - Retirar um registro específico.
  - Ordenar um arquivo para obter todos os registros em ordem de acordo com a chave.
  - Juntar dois arquivos para formar um arquivo maior.

# Dicionário

- Nome comumente utilizado para descrever uma estrutura de dados para pesquisa
- Dicionário é um tipo abstrato de dados com as operações:
  - Inicializa
  - Pesquisa
  - Insere
  - Retira

# Dicionário

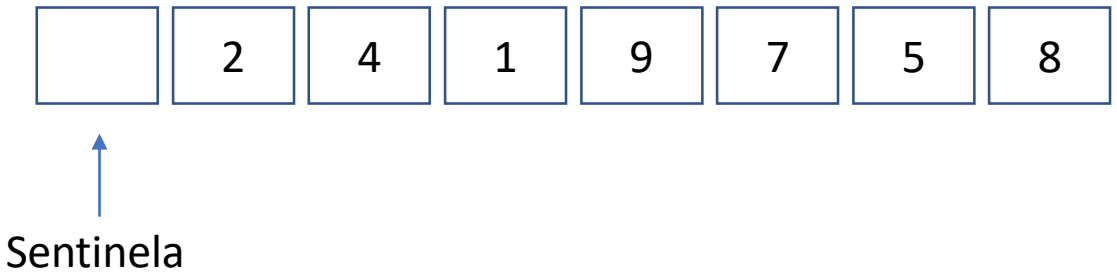
| Chave | | Registro |
|---|---|---|
| 1 | → | Ana |
| 2 | → | Patrícia |
| 3 | → | Amanda |
| 4 | → | Alberto |
| 5 | → | Pedro |
| 6 | → | Laura |

# Pesquisa Sequencial

- Método de pesquisa mais simples:
  - a partir do primeiro registro, pesquise sequencialmente até encontrar a chave procurada; então pare.
- Armazenamento de um conjunto de registros por meio do tipo estruturado arranjo

# Pesquisa Sequencial

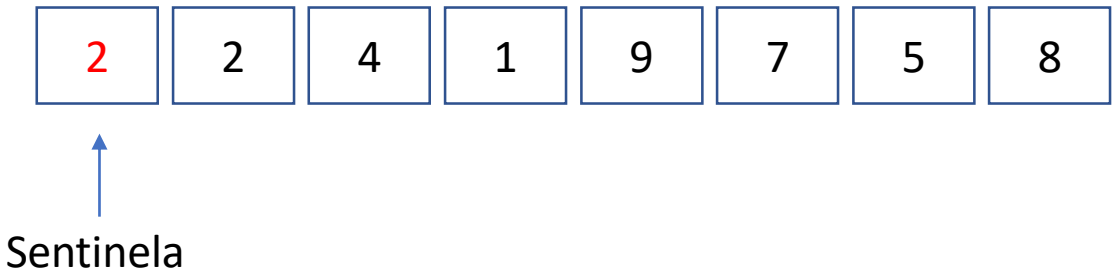
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

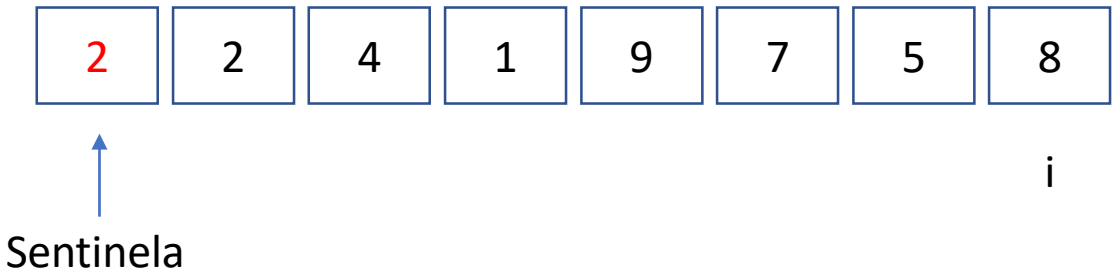
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

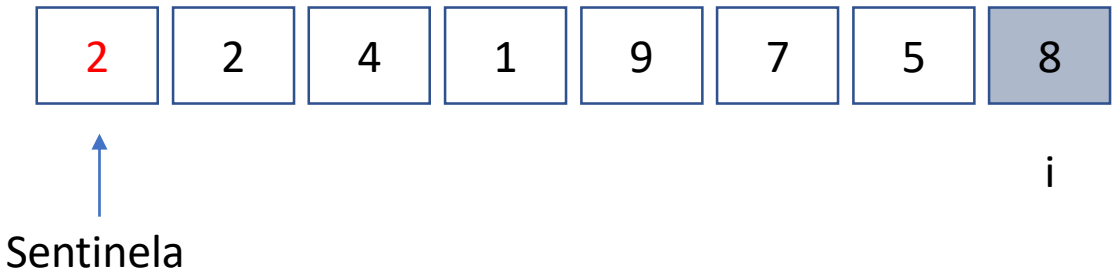
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

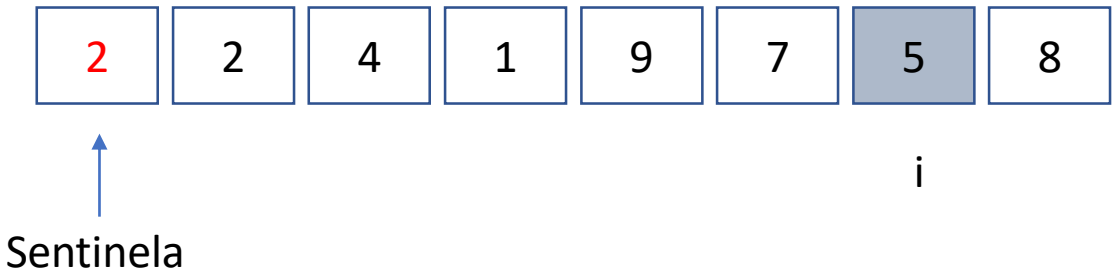
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

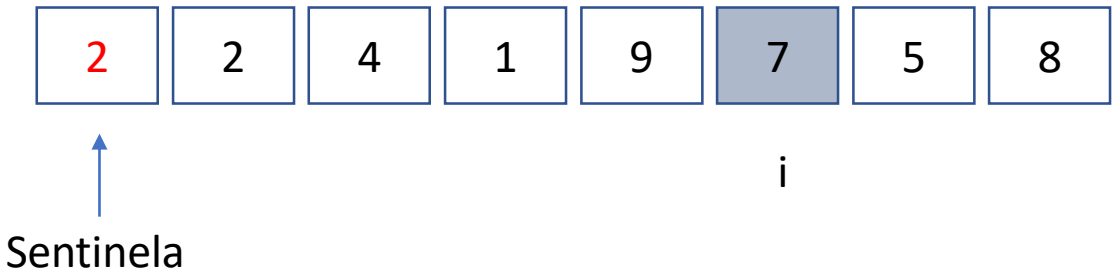
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

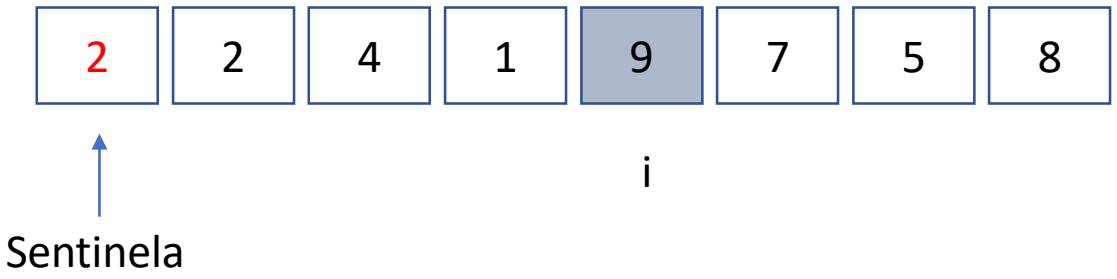
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

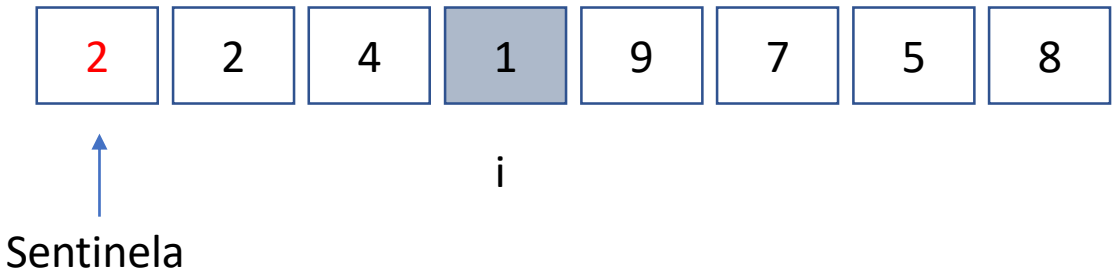
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

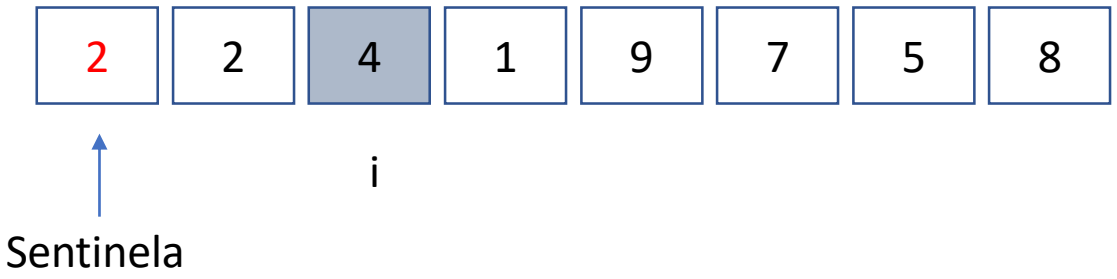
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

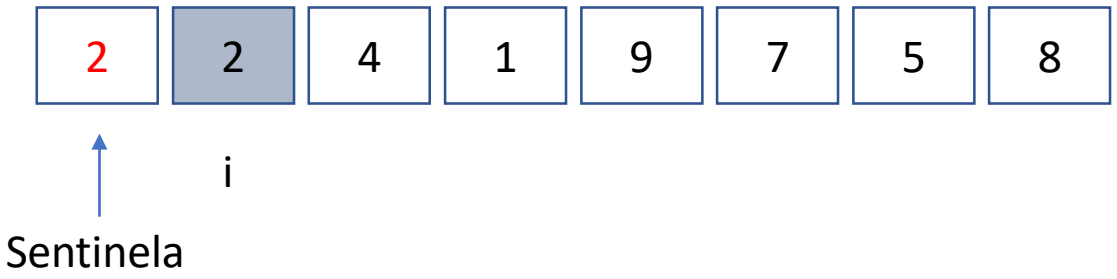
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

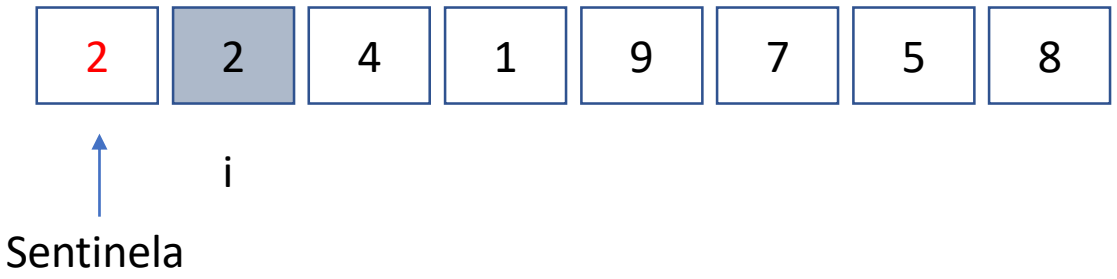
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

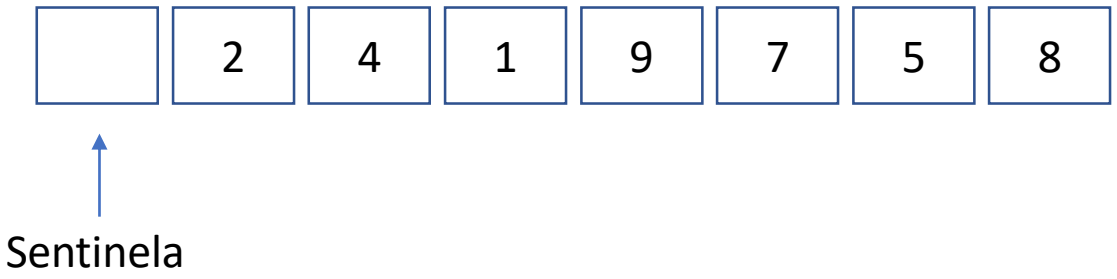
Chave pesquisa: reg = 2 – pior caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

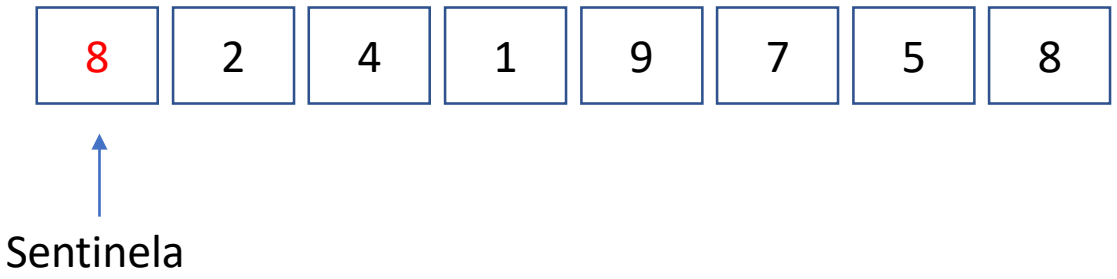
Chave pesquisa: reg = 8 – Melhor caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

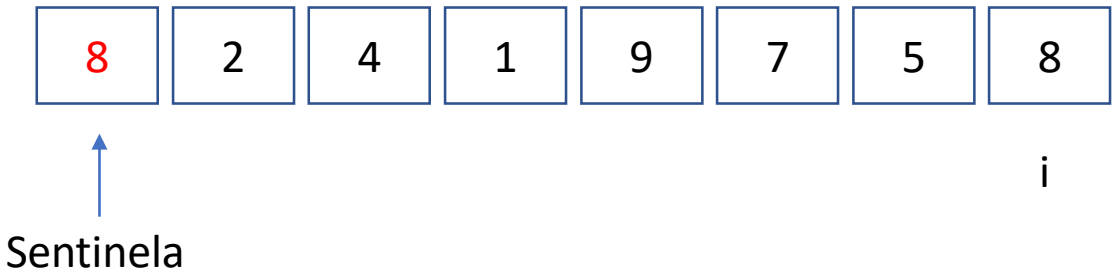
Chave pesquisa: reg = 8 – Melhor caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

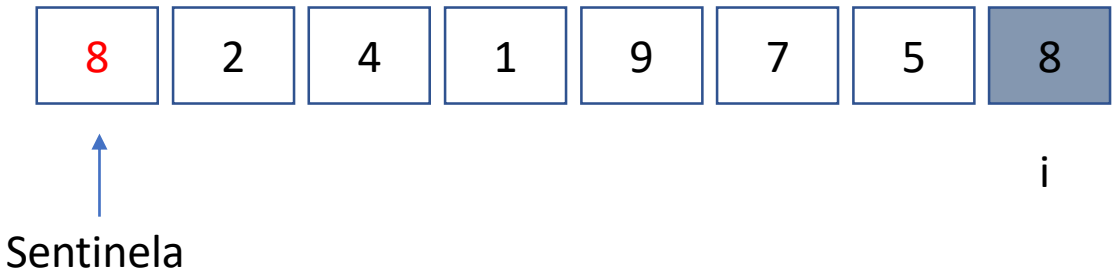
Chave pesquisa: reg = 8 – Melhor caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

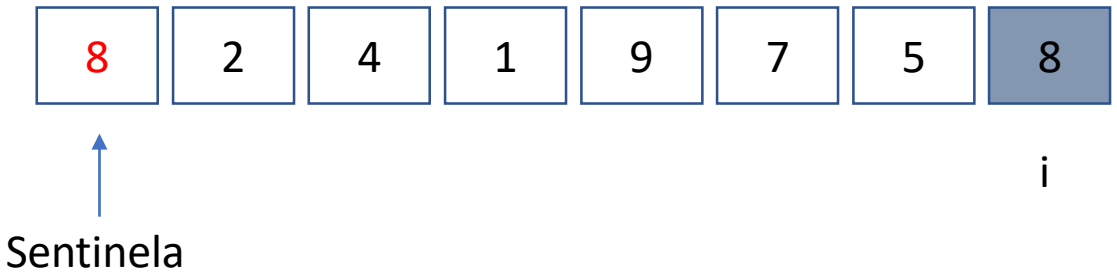
Chave pesquisa: reg = 8 – Melhor caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

Chave pesquisa: reg = 8 – Melhor caso




```
public int pesquisa(Item reg) {
    this.registros[0] = reg; // sentinela
    int i = this.n;
    while (this.registros[i].chave() != reg.chave())
        i--;
    return i;
}
```

# Pesquisa Sequencial

```
public class Tabela {
    private Item registros[];
    private int n;

    public Tabela(int maxN) {
        this.registros = new Item[maxN + 1];
        this.n = 0;
    }

    public int pesquisa(Item reg) {
        this.registros[0] = reg; // sentinela
        int i = this.n;
        while (this.registros[i].chave() != reg.chave())
            i--;
        return i;
    }

    public void insere(Item reg) throws Exception {
        if (this.n == (this.registros.length - 1))
            throw new Exception("Erro: A tabela esta cheia");
        this.registros[++this.n] = reg;
    }
}
```

# Pesquisa Sequencial

- Cada registro contém um campo chave que identifica o registro.
- Além da chave, podem existir outros componentes em um registro, os quais não têm influência nos algoritmos.
- O método pesquisa retorna o índice do registro que contém a chave passada como parâmetro no registro reg; caso não esteja presente, o valor retornado é zero.

# Pesquisa Sequencial

- Utilização de um registro sentinela na posição zero do vetor:
  - Garante que a pesquisa sempre termina: se o índice retornado por Pesquisa for zero, a pesquisa foi sem sucesso.
  - Não é necessário testar se i > 0, devido a isto:
    - o índice i é decrementado e a chave de pesquisa é comparada com a chave que está no registro.

# Pesquisa Sequencial

- Análise de custo:
  - Pesquisa com sucesso:
    - melhor caso: $C(n) = 1$
    - pior caso: $C(n) = n$
    - caso médio: $C(n) = (n + 1)/2$
  - Pesquisa sem sucesso:
    - $C(n) = n + 1$

# Exercícios

- Implemente uma versão da busca sequencial para encontrar a chave $x$ em um vetor de $n$ elementos. Faça com que seu programa retorne também quantas operações de comparação foram necessárias para recuperar a chave pesquisada.

# Pesquisa Binária

- Pesquisa em tabela pode ser mais eficiente
  - Desde que registros sejam mantidos em ordem.
- Para saber se uma chave está presente na tabela:
  - Compare a chave com o registro que está na posição do meio da tabela.
  - **Se** a chave é menor **então** o registro procurado está na primeira metade da tabela.
  - **Se** a chave é maior **então** o registro procurado está na segunda metade da tabela.

# Pesquisa Binária

- Repita o processo até que a chave seja encontrada, ou fique apenas um registro cuja chave é diferente da procurada, significando uma pesquisa sem sucesso.

# Pesquisa Binária

Chave pesquisa G

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | E | F | G | H |
| | esq | | | | | | | dir |

```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

Chave pesquisa G

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | E | F | G | H |
| | esq | | | i | | | | dir |

```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

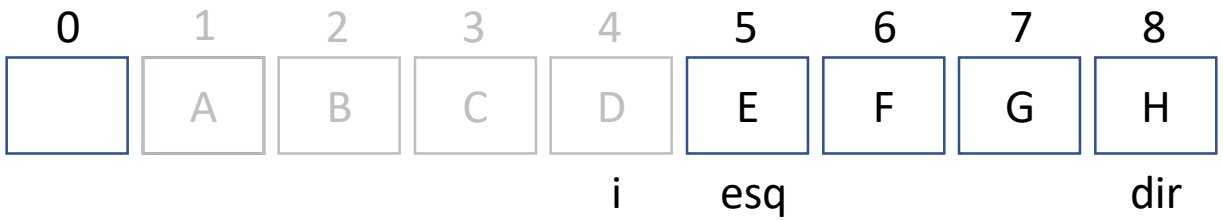
Chave pesquisa G

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | E | F | G | H |
| | esq | | | i | | | | dir |

```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

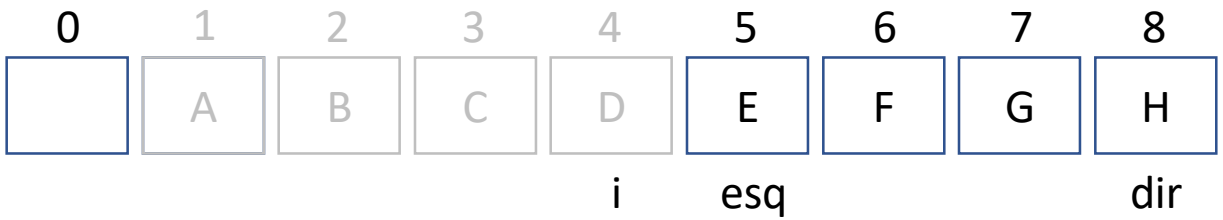
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

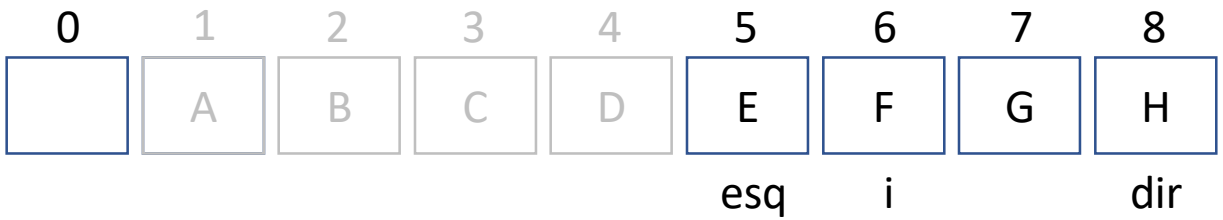
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

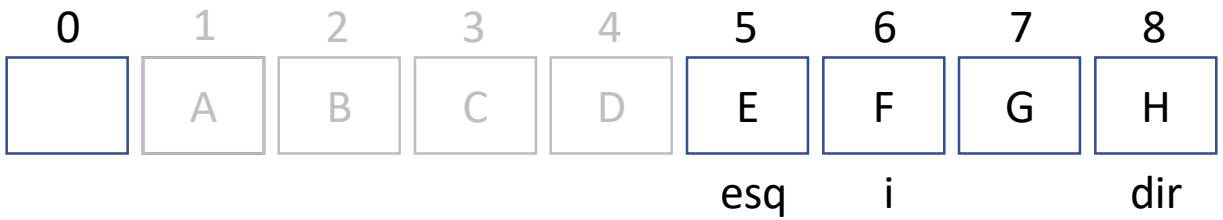
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

Chave pesquisa G

```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

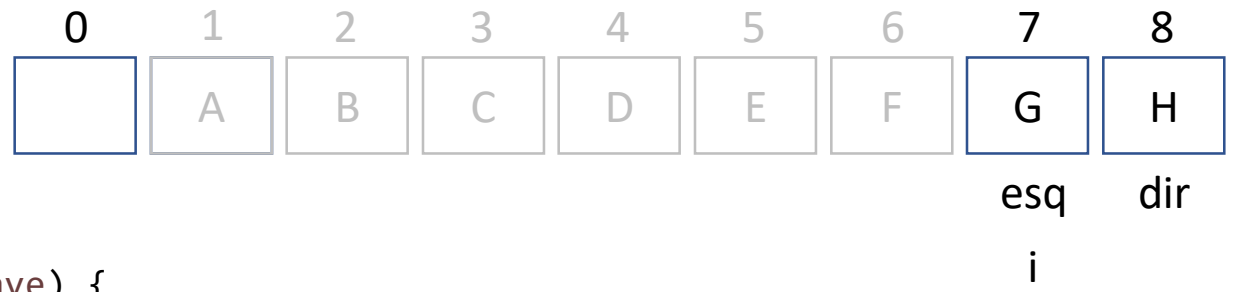
# Pesquisa Binária

Chave pesquisa G

```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

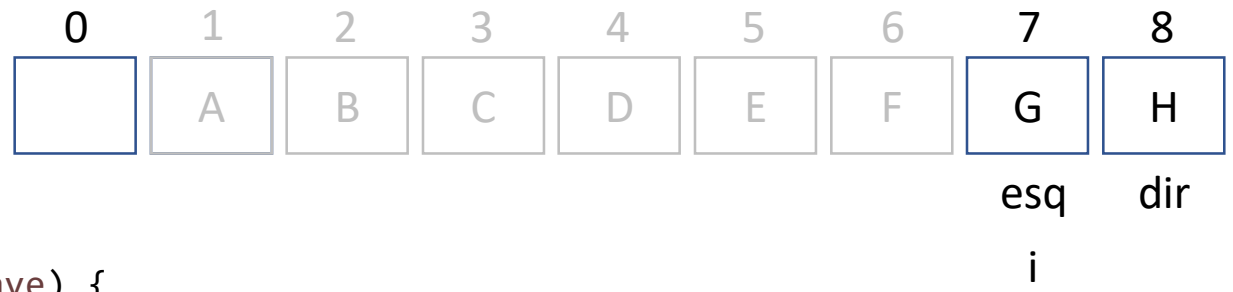
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

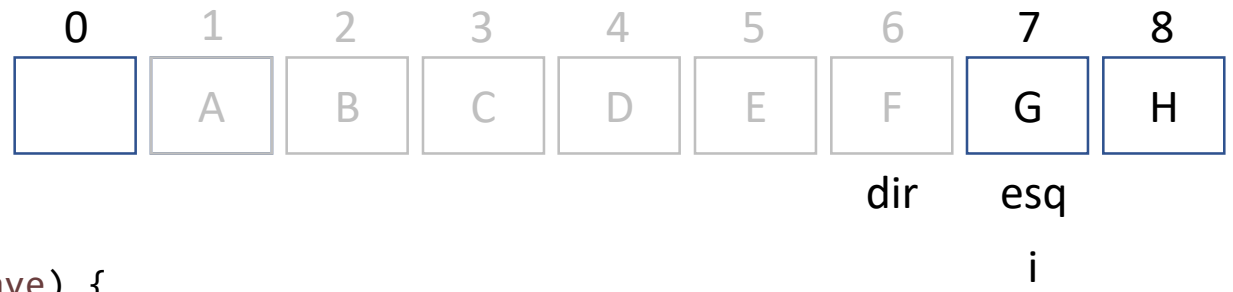
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

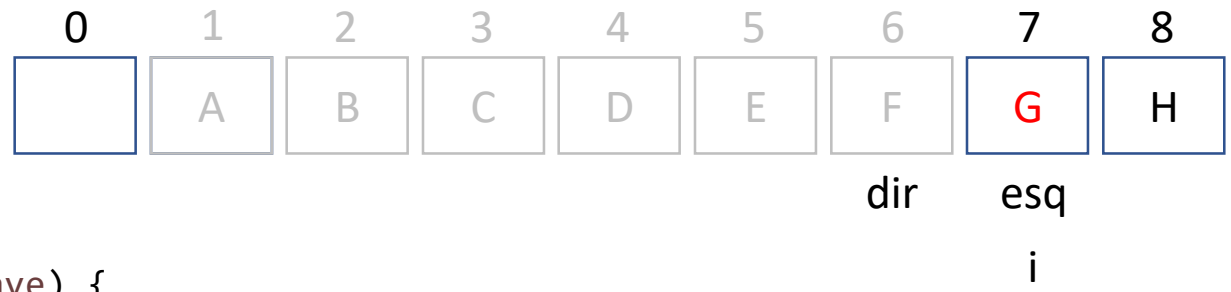
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

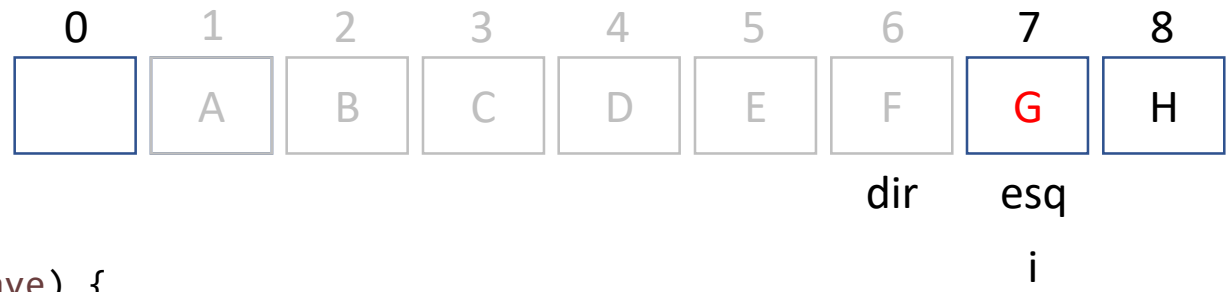
Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

Chave pesquisa G




```
public int binaria(Item chave) {
    if (this.n == 0) return 0;
    int esq = 1, dir = this.n, i;
    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));
    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

```
public int binaria(Item chave) {
    if (this.n == 0)
        return 0;
    int esq = 1, dir = this.n, i;

    do {
        i = (esq + dir) / 2;
        if (chave.chave() > this.registros[i].chave())
            esq = i + 1;
        else
            dir = i - 1;
    } while ((chave.chave() != this.registros[i].chave()) && (esq <= dir));

    if (chave.chave() == this.registros[i].chave())
        return i;
    else
        return 0;
}
```

# Pesquisa Binária

- Análise:
  - A cada iteração do algoritmo, o tamanho da tabela é dividido ao meio.
  - Número de vezes que o tamanho da tabela é dividido ao meio é cerca de *log n*.
  - Entretanto, o custo para manter a tabela ordenada é alto:
    - a cada inserção na posição $p$ da tabela implica no deslocamento dos registros a partir da posição $p$ para as posições seguintes.
  - Não deve ser usada para aplicações muito dinâmicas.

# Exercícios

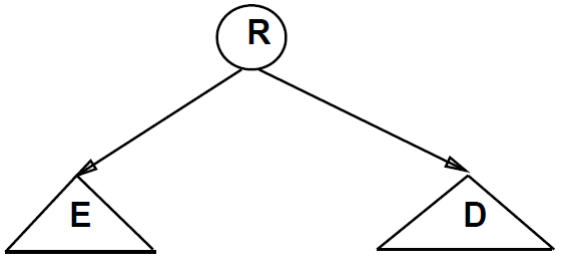
- Repita o mesmo procedimento do exercício realizado anteriormente (utilizando busca sequencial), mas agora utilizando busca binária.
- Retorne também quantas operações de comparação foram necessárias para recuperar a chave pesquisada.
- Compare o resultado o obtido na busca binária com a busca sequencial.
- Lembre-se que para busca binária, o vetor deve estar ordenado, para isto, faça um procedimento para sempre inserir uma nova chave de forma sempre ordenada no vetor ou ordene o vetor após a inserção de todos os elementos (utilize os algoritmos de ordenação estudados em sala).

# Árvores de Pesquisa

- Estrutura de dados muito eficiente para armazenar informação.
- Adequada quando existe necessidade de considerar todos ou alguma combinação de:
  - Acessos eficientes.
  - Facilidade de inserção e retirada de registros.
  - Boa taxa de utilização de memória.
  - Utilização de memória primária e secundária.

# Árvores de Pesquisa

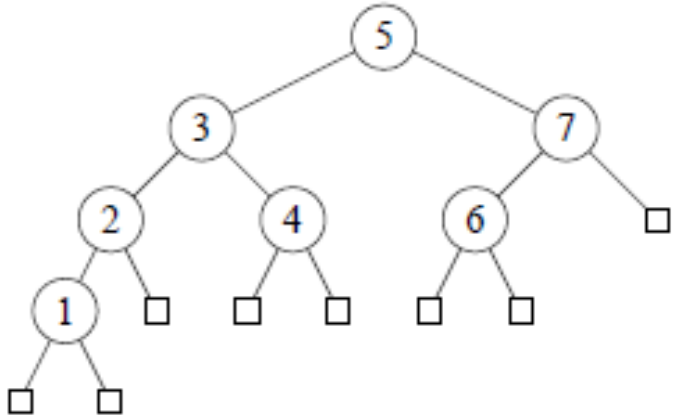
- Para qualquer nó que contenha um registro:




- Tem-se a relação invariante:

  - Todos os registros com chaves **menores** estão na sub árvore à **esquerda**.
  - Todos os registros com chaves **maiores** estão na sub árvore à **direita**.

# Árvores de Pesquisa sem balanceamento




- O **nível** do nó raiz é 0.
- Se um nó está no **nível** $i$ então a raiz de suas sub árvores estão no **nível** $i + 1$.
- A **altura** de um nó é o comprimento do caminho mais longo deste nó até um nó folha.
- A **altura** de uma árvore é a altura do nó raiz.

# Árvores de Pesquisa – alguns conceitos

- Raiz da árvore: nó que não tem antecedente
- Folha: nós que não tem descendentes (os outros são os nós internos)
- A altura de um nó na árvore é o caminho de maior comprimento que se pode fazer deste nó a uma folha.
- A altura da árvore é a altura de sua raiz.
- A profundidade de um nó é o comprimento da raiz até o nó (só existe um caminho)

# Implementação do TAD Dicionário usando Árvore Binária de Pesquisa

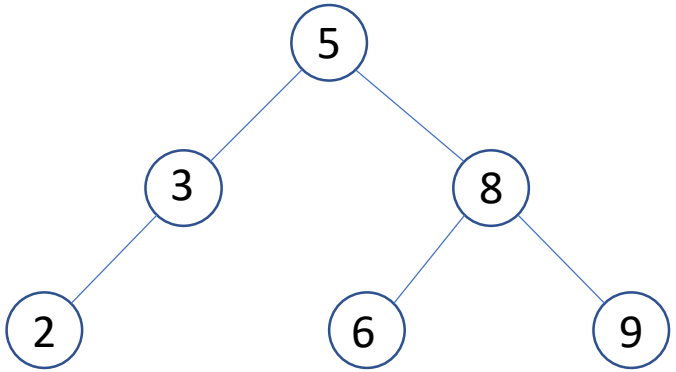
- Estrutura de dados:
  - Contém as operações
    - Inicializa
    - Pesquisa
    - Insere
    - retira.
  - A operação *inicializa* é implementada pelo construtor da classe *ArvoreBinaria*.

# Método para Pesquisar na Árvore

- Para encontrar um registro com uma chave *reg*:
  - Compare-a com a chave que está na raiz .
  - Se é menor, vá para a sub árvore esquerda.
  - Se é maior, vá para a sub árvore direita.
  - Repita o processo recursivamente, até que a chave procurada seja encontrada ou um nó folha é atingido.
  - Se a pesquisa tiver sucesso então o registro contendo a chave passada em *reg* é retornado.

# Método para Pesquisar na Árvore

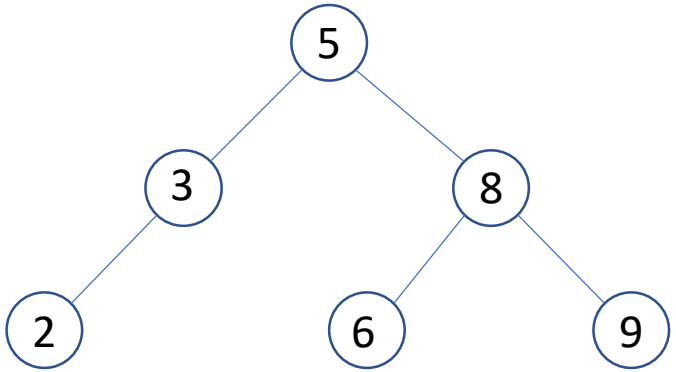
reg = 6




```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

# Método para Pesquisar na Árvore

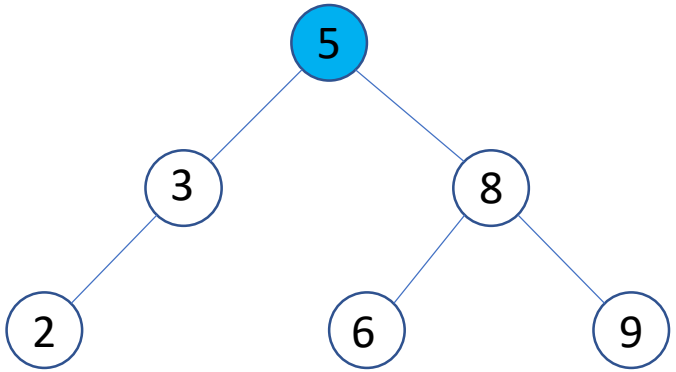
reg = 6




```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

# Método para Pesquisar na Árvore

reg = 6

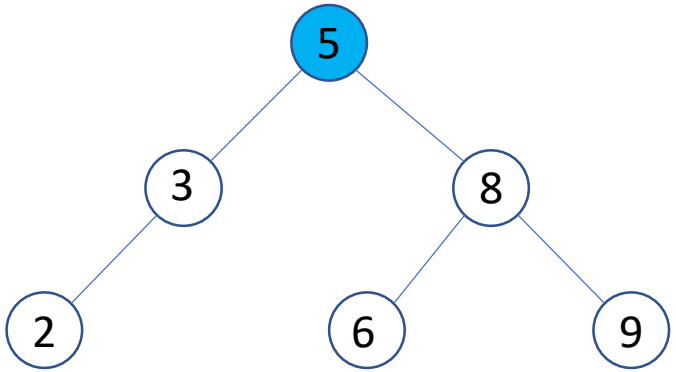



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.*

# Método para Pesquisar na Árvore

reg = 6

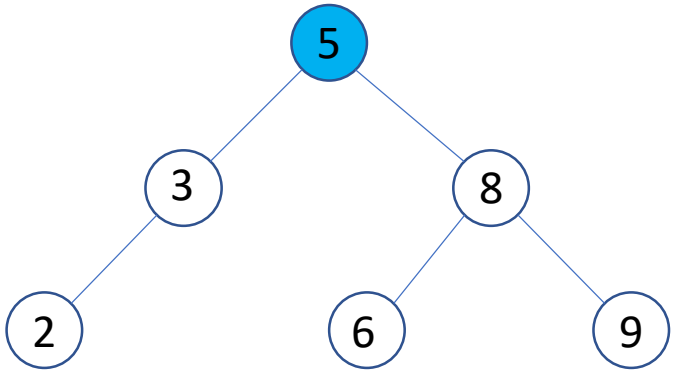



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Se for maior, vá para a sub árvore direita.*

# Método para Pesquisar na Árvore

reg = 6

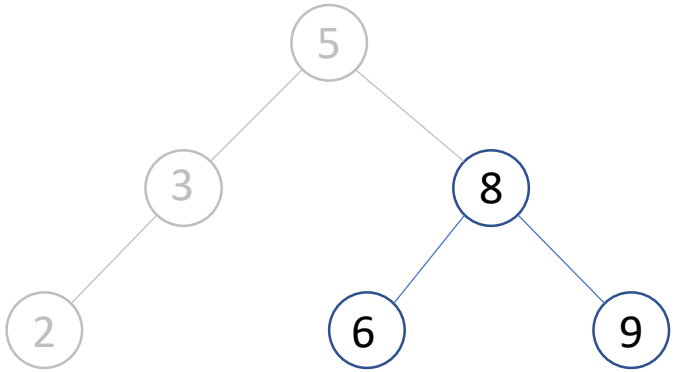



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Repita o processo recursivamente, passando para a função apenas a sub árvore direita*

# Método para Pesquisar na Árvore

reg = 6

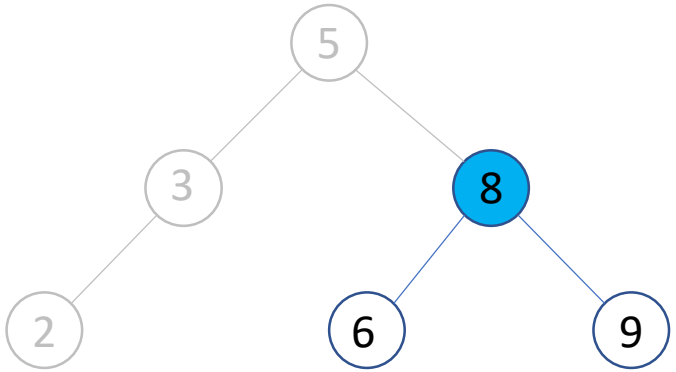



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*O No p é a sub árvore direita.*

# Método para Pesquisar na Árvore

reg = 6

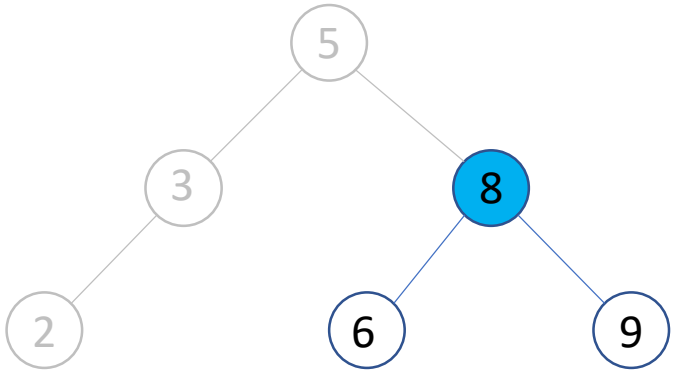



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.*

# Método para Pesquisar na Árvore

reg = 6

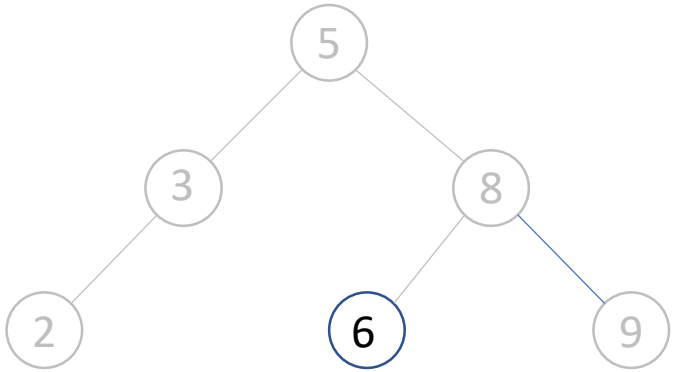



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Repita o processo recursivamente, passando para a função apenas a sub árvore esquerda*

# Método para Pesquisar na Árvore

reg = 6

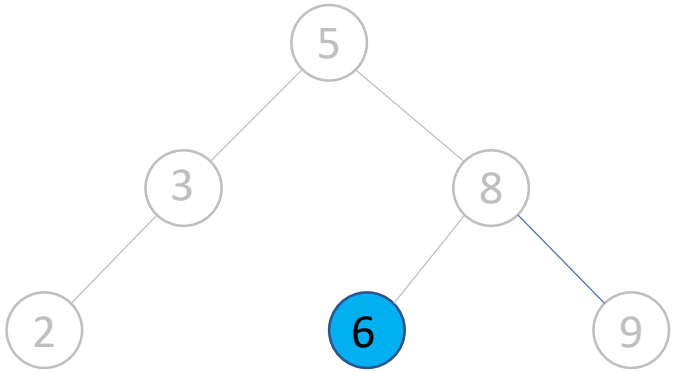



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*O No p é a sub árvore esquerda.*

# Método para Pesquisar na Árvore

reg = 6

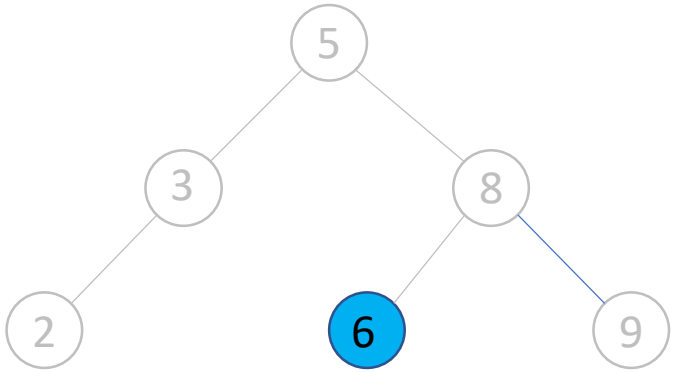



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.*

# Método para Pesquisar na Árvore

reg = 6

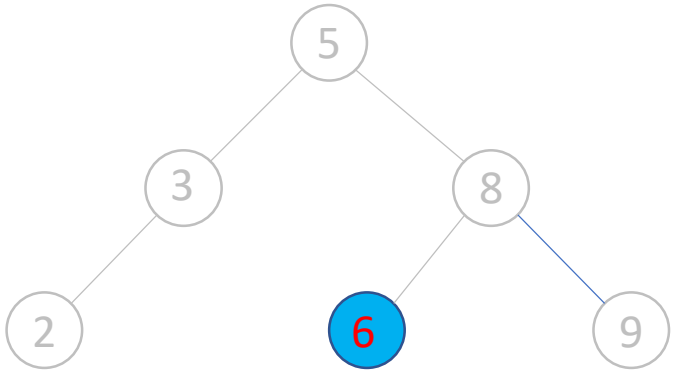



```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

*Compare reg com a chave que está na raiz. Se for maior, vá para a sub árvore direita.*

# Método para Pesquisar na Árvore

reg = 6




```
private Item pesquisa(Item reg, No p) {
    if (p == null)
        return null; // Registro não econtrado
    else if (reg.chave() < p.reg.chave())
        return pesquisa(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        return pesquisa(reg, p.dir);
    else
        return p.reg;
}
```

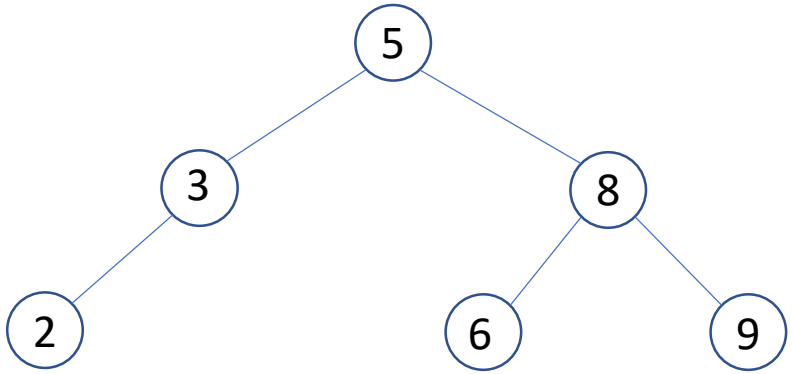
*Pesquisa obteve sucesso: retorne o registro contendo a chave reg.*

# Procedimento para Inserir na Árvore

- Ao pesquisar por um registro, atingir uma referência **null** em um processo de pesquisa significa uma pesquisa sem sucesso.
- Caso queira inseri-lo na árvore, a referência **null** atingida é justamente o ponto de inserção.

# Procedimento para Inserir na Árvore

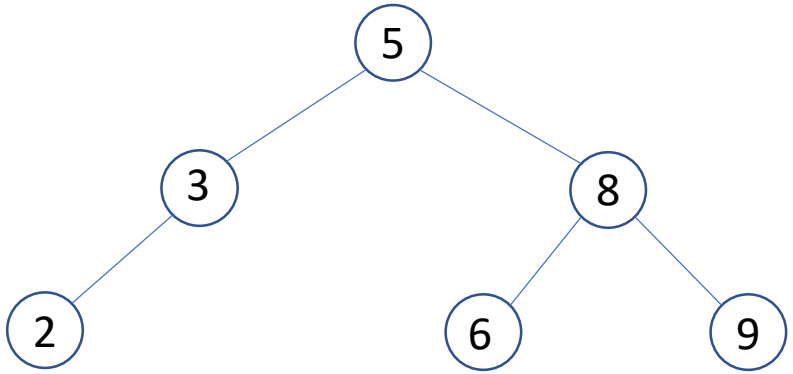
reg = 4




```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

# Procedimento para Inserir na Árvore

reg = 4

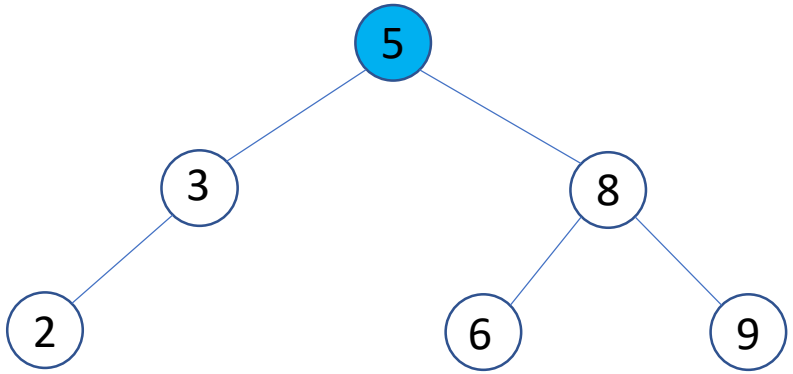



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Se p = null, árvore está vazia, então cria-se um novo No p para a árvore, que conterá apenas um elemento.*

# Procedimento para Inserir na Árvore

reg = 4

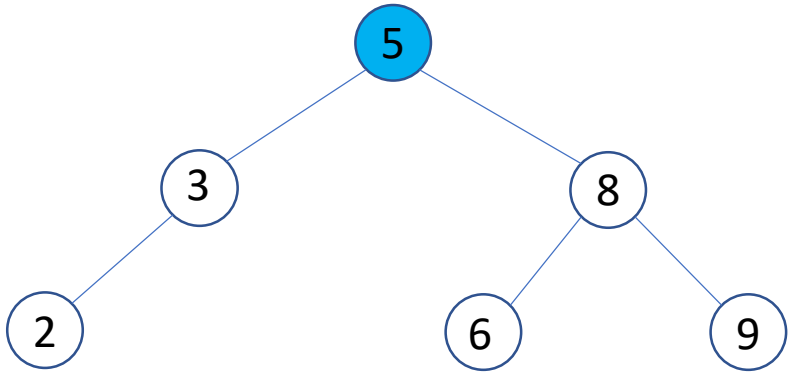



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.*

# Procedimento para Inserir na Árvore

reg = 4

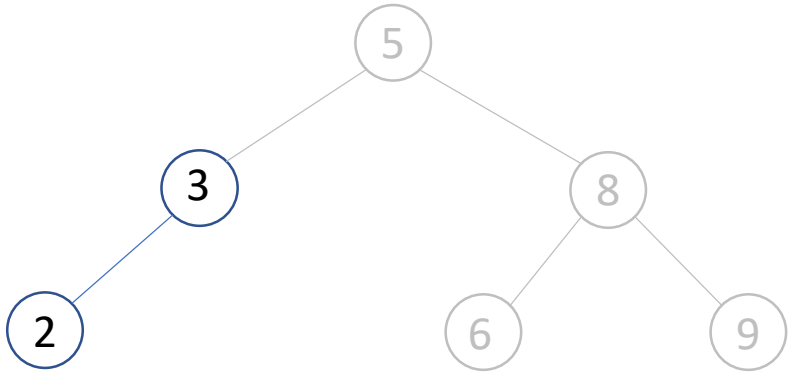



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Repita o processo recursivamente, passando para a função apenas a sub árvore esquerda*

# Procedimento para Inserir na Árvore

reg = 4

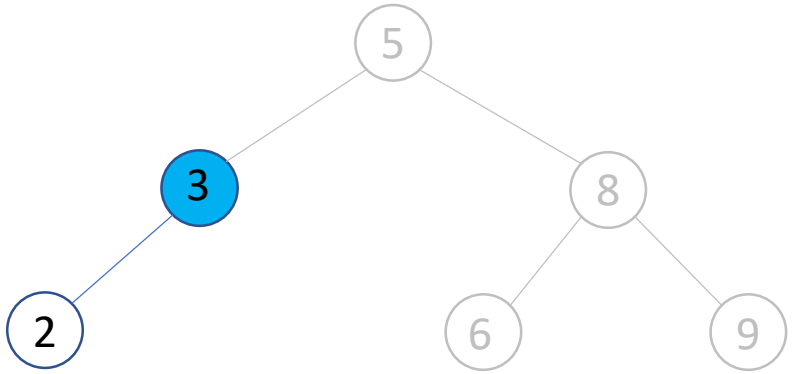



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Se p = null, árvore está vazia, então cria-se um novo No p para a árvore, que conterá apenas um elemento.*

# Procedimento para Inserir na Árvore

reg = 4

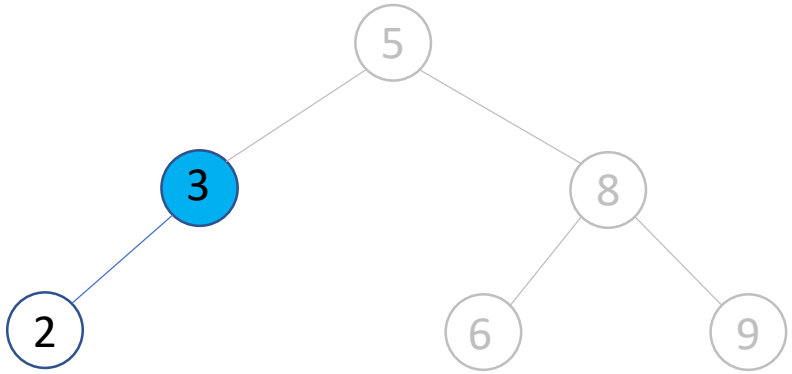



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Compare reg com a chave que está na raiz. Se for menor, vá para a sub árvore esquerda.*

# Procedimento para Inserir na Árvore

reg = 4

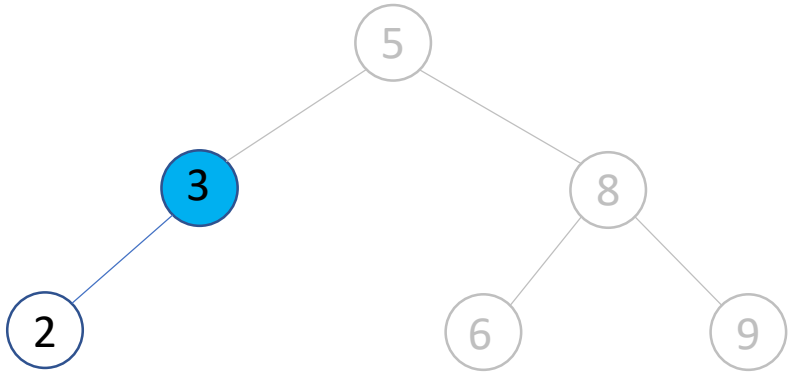



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Compare reg com a chave que está na raiz. Se for maior, vá para a sub árvore direita.*

# Procedimento para Inserir na Árvore

reg = 4

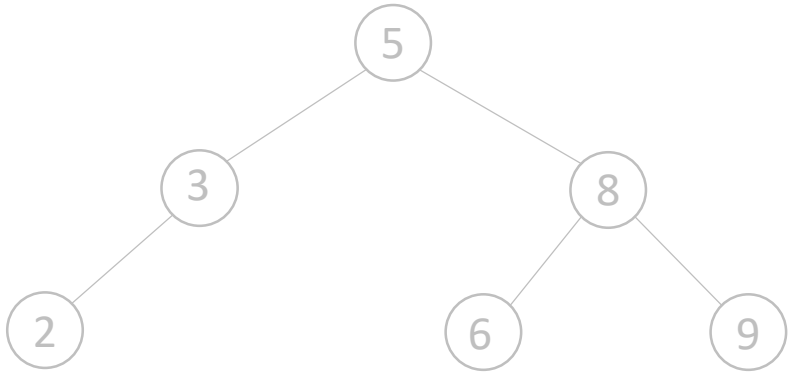



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Repita o processo recursivamente, passando para a função apenas a sub árvore direita.*
*Obs.: Neste momento, p.dir = null*

# Procedimento para Inserir na Árvore

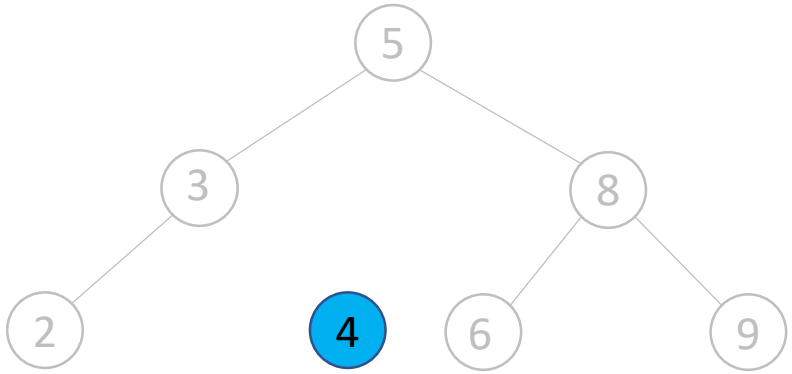
reg = 4




```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

# Procedimento para Inserir na Árvore

reg = 4

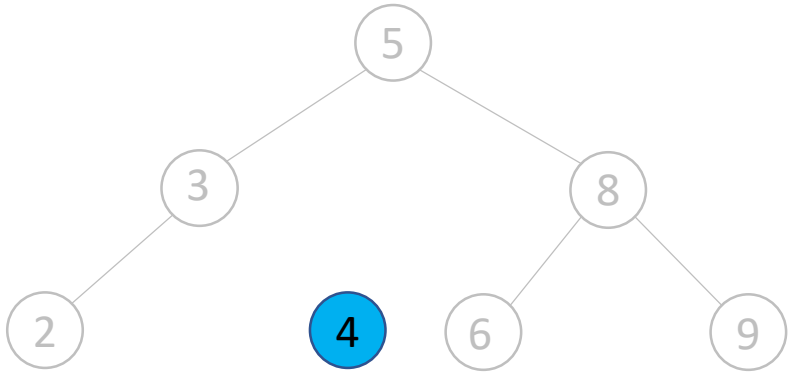



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Cria-se um novo nó, e insere o registro nele*

# Procedimento para Inserir na Árvore

reg = 4

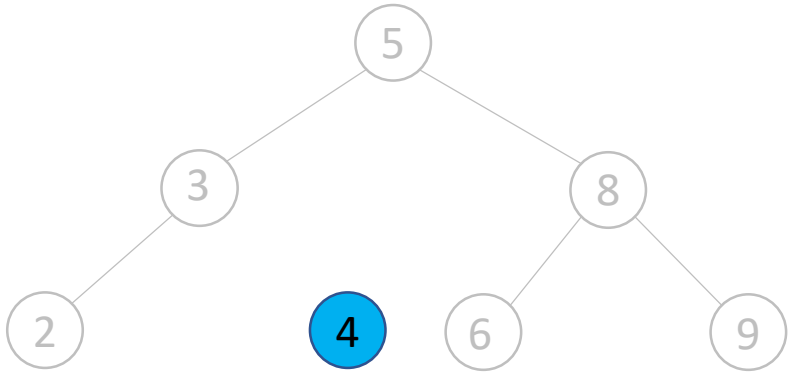



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Como o novo nó inserido não tem nós filhos, as sub árvores esquerda e direita serão nulas*

# Procedimento para Inserir na Árvore

reg = 4

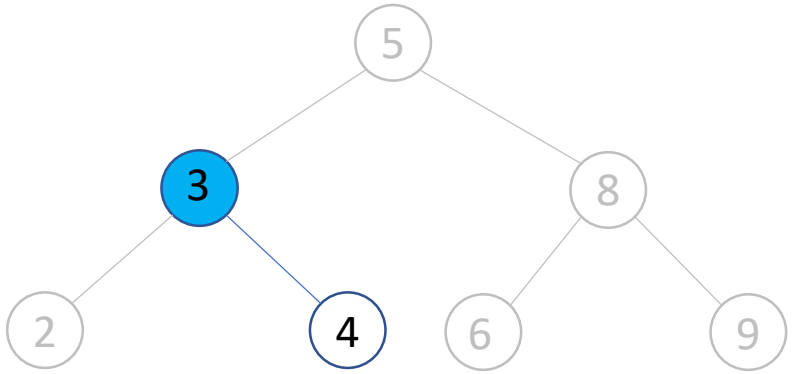



```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*Retorna o novo nó para a função.*

# Procedimento para Inserir na Árvore

reg = 4



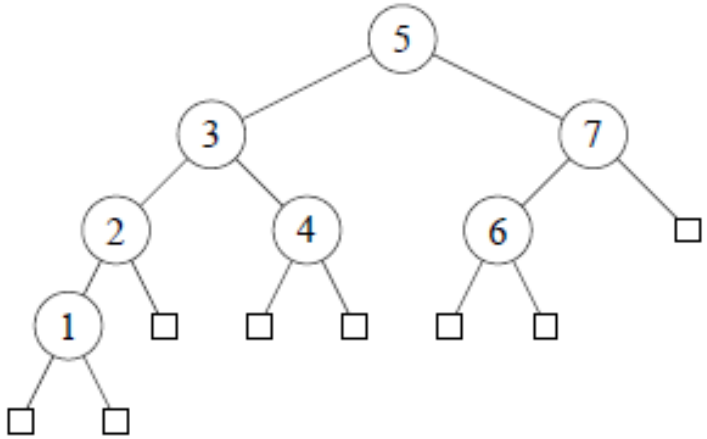

```
private No insere(Item reg, No p) {
    if (p == null) {
        p = new No(); p.reg = reg;
        p.esq = null; p.dir = null;
    }
    else if (reg.chave() < p.reg.chave())
        p.esq = insere(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = insere(reg, p.dir);
    else
        System.out.println("Erro: Registro ja existente");
    return p;
}
```

*A sub árvore direita do nó (3) recebe o novo nó (4) como nó filho.*

# Procedimento para Retirar um elemento *x* da Árvore

- A retirada de um registro não é tão simples quanto a inserção.
- Se o nó que contém o registro a ser retirado possui no máximo um descendente: a operação é simples.
- No caso do nó conter dois descendentes o registro a ser retirado deve ser primeiro:
  - substituído pelo **registro mais à direita na sub árvore esquerda**;
  - ou pelo **registro mais à esquerda na sub árvore direita**.

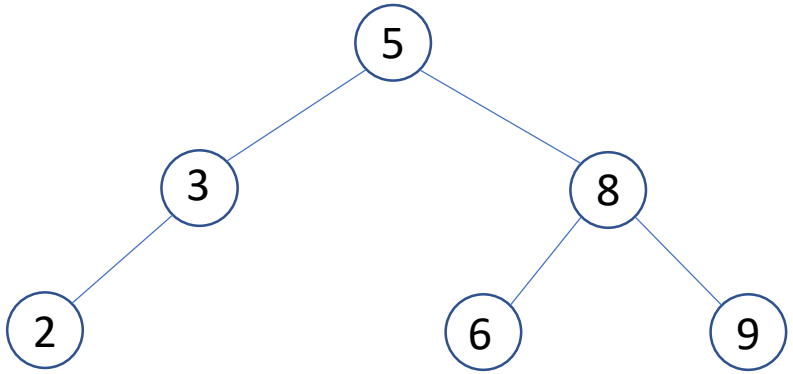
# Procedimento para Retirar um elemento *x* da Árvore




- Retirar o registro com chave 5:
  - trocá-lo pelo registro com chave 4 (**registro mais à direita na sub árvore esquerda**) ou pelo registro com chave 6 (**registro mais à esquerda na sub árvore direita**), e então retirar o nó que recebeu o registro com chave 5

# Procedimento para Retirar um elemento *x* da Árvore

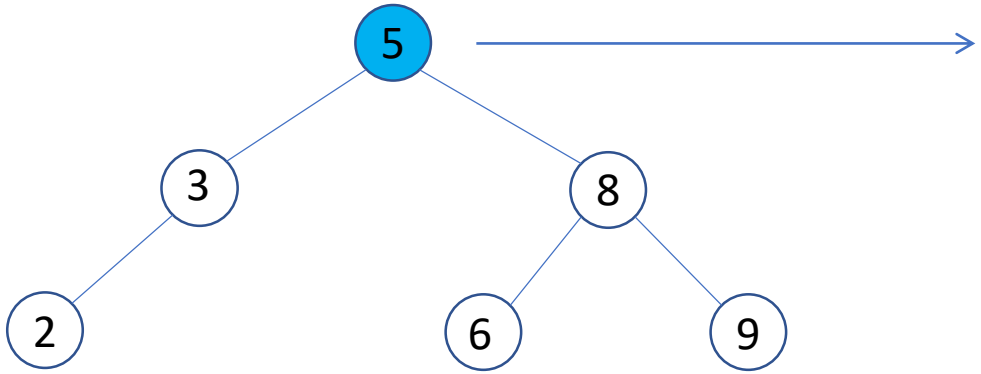
reg = 5




```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

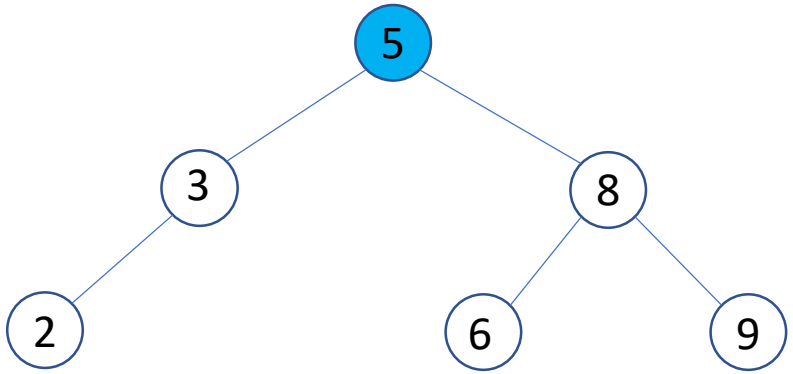



*Pesquisa o elemento a ser removido na árvore*

```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

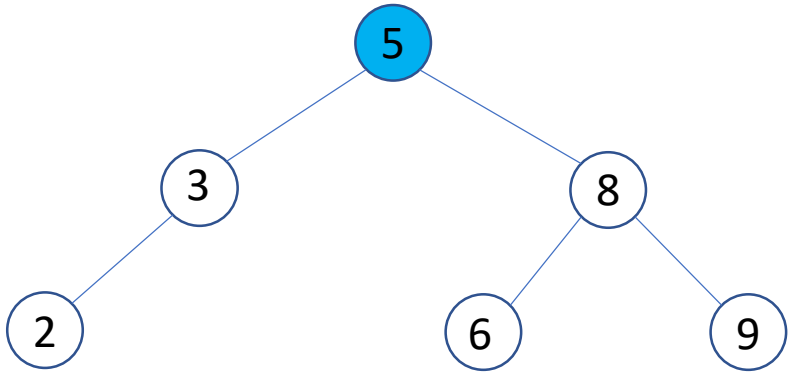



```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

*Se não existir a sub árvore dir, a raiz da sub árvore esq passa a ser a raiz da árvore.*

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

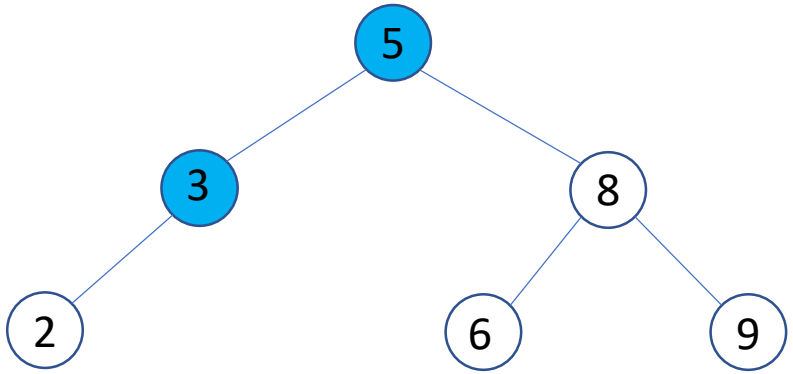



```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

*Se não existir a sub árvore esq, a raiz da sub árvore dir passa a ser a raiz da árvore.*

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

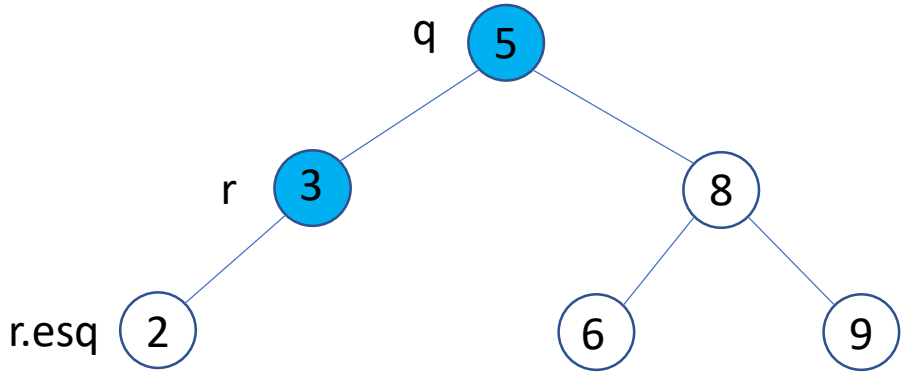



```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

*Caso exista sub árvore esq e dir, calcular o elemento mais a direta da sub árvore esquerda de p*

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

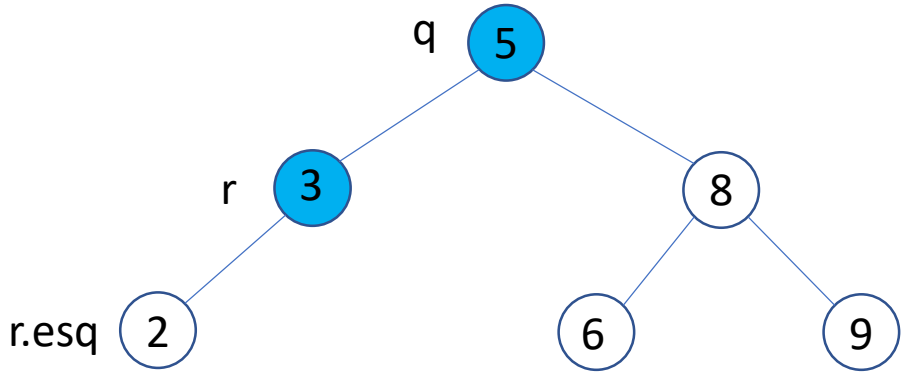



*Se existe uma sub árvore direita, continuar pesquisa até encontrar o elemento mais a direita*

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

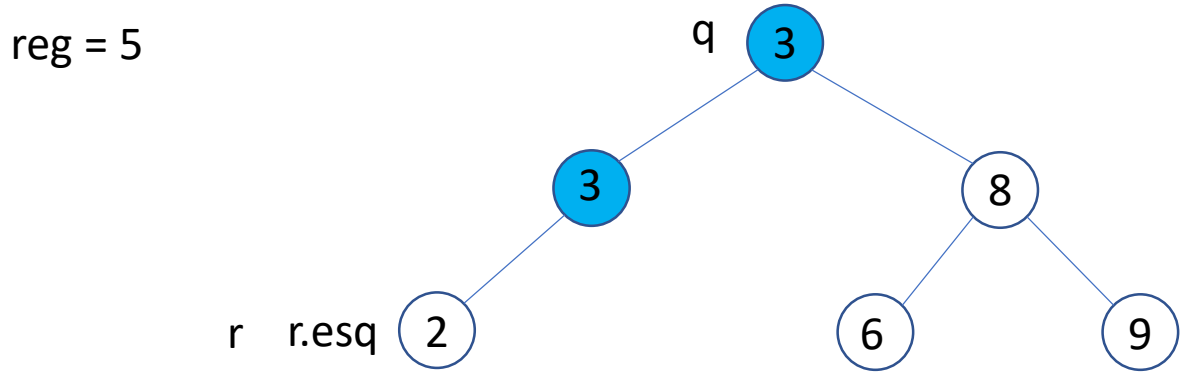
reg = 5




*Se existe uma sub árvore direita, continuar pesquisa até encontrar o elemento mais a direita*

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

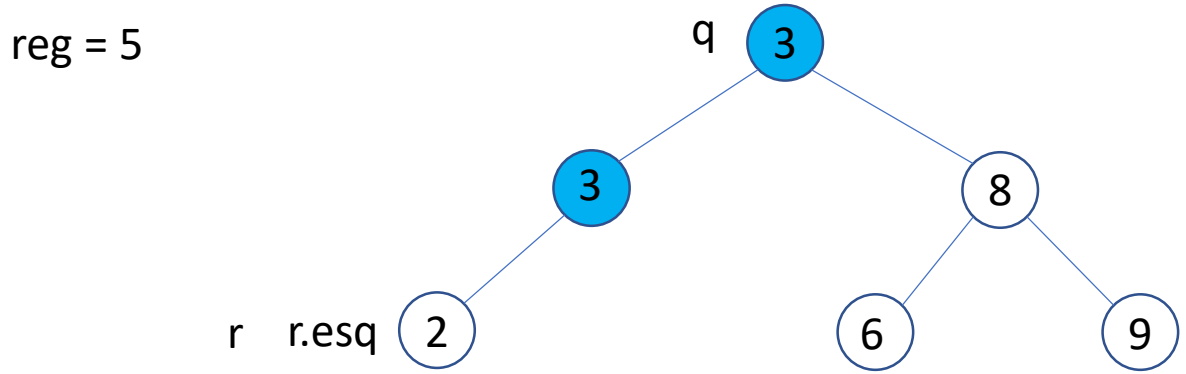
# Procedimento para Retirar um elemento *x* da Árvore




*Ao encontrar o elemento mais a direita, q recebe r (o elemento mais a direita), e r recebe sua sub árvore a esquerda*

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

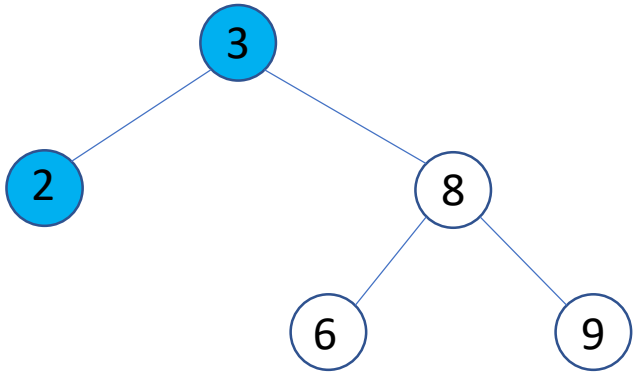



*Retorna a sub árvore a esquerda*

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5

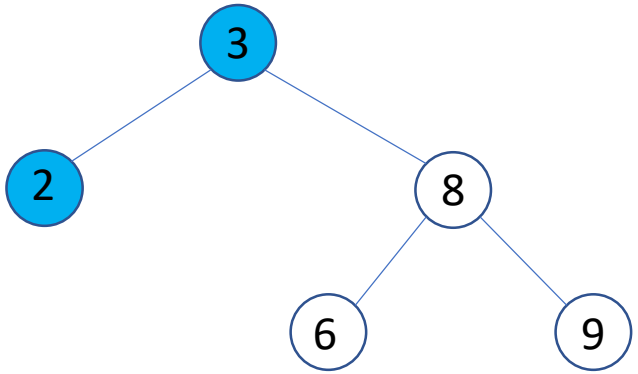



```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

reg = 5




```
private No retira(Item reg, No p) {
    ...
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null) p = p.esq;
        else if (p.esq == null) p = p.dir;
        else p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

Retorna a nova raiz da árvore

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Procedimento para Retirar um elemento *x* da Árvore

```
private No retira(Item reg, No p) {
    if (p == null)
        System.out.println("Erro: Registro nao encontrado");
    else if (reg.chave() < p.reg.chave())
        p.esq = retira(reg, p.esq);
    else if (reg.chave() > p.reg.chave())
        p.dir = retira(reg, p.dir);
    else {
        if (p.dir == null)
            p = p.esq;
        else if (p.esq == null)
            p = p.dir;
        else
            p.esq = antecessor(p, p.esq);
    }
    return p;
}
```

```
private No antecessor(No q, No r) {
    if (r.dir != null)
        r.dir = antecessor(q, r.dir);
    else {
        q.reg = r.reg;
        r = r.esq;
    }
    return r;
}
```

# Caminhamento Central (ou Em-ordem)

- Após construída a árvore, pode ser necessário percorrer todos os registros que compõem a tabela ou arquivo.
- Existe mais de uma ordem de **caminhamento** em árvores, mas a mais útil é a chamada ordem de **caminhamento central**.
- O caminhamento central é melhor expresso em termos recursivos.

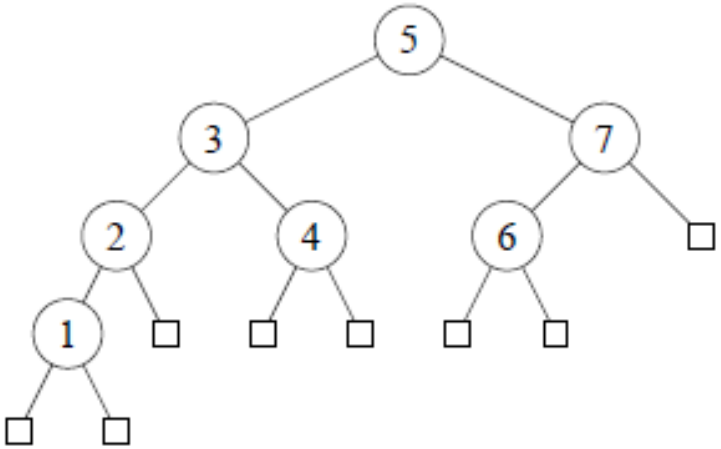
# Caminhamento Central

- O caminhamento central lista o **nó filho da subárvore mais à esquerda**, lista o **nó raiz** desta subárvore e então o **nó filho à direita**. (em cada subárvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **nós filhos à esquerda**, **raiz** e **nós filhos à direita**.

- Uma característica importante do caminhamento central é que os nós são visitados de **forma ordenada**.

# Caminhamento Central

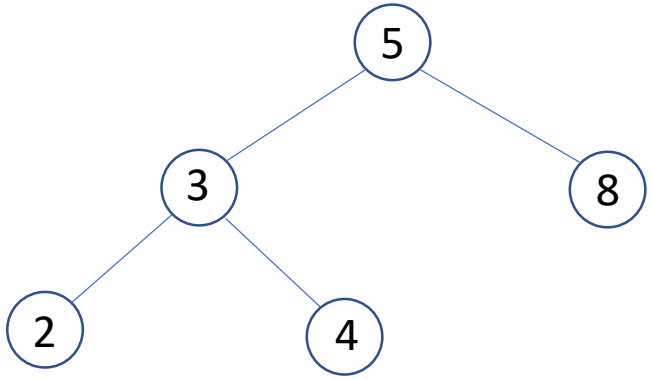
- Percorrer a árvore:




- Usando caminhamento central recupera as chaves na ordem 1, 2, 3, 4, 5, 6 e 7.

# Caminhamento Central

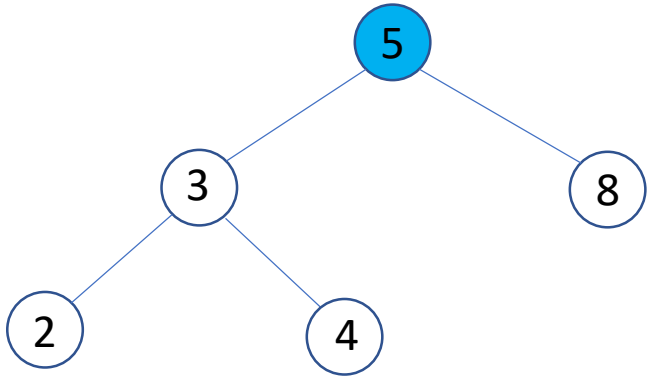
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

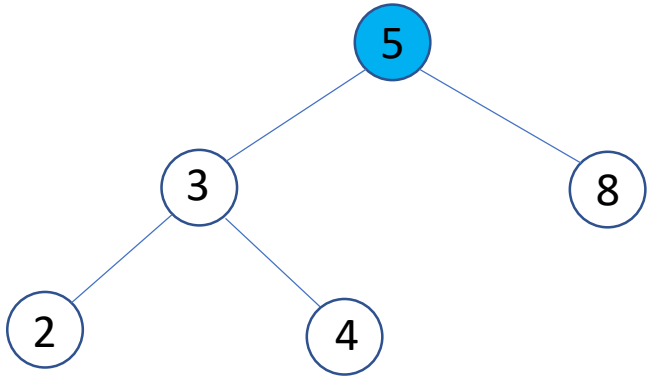
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

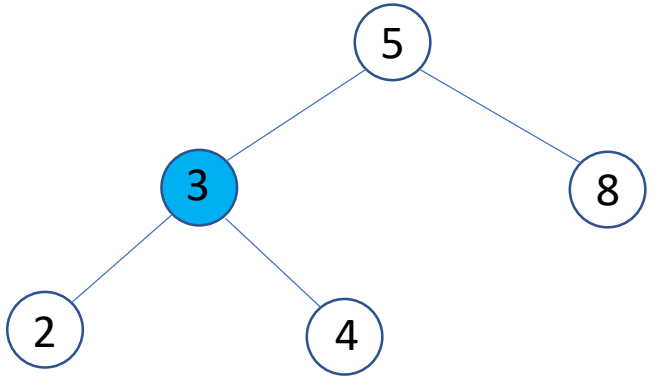
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

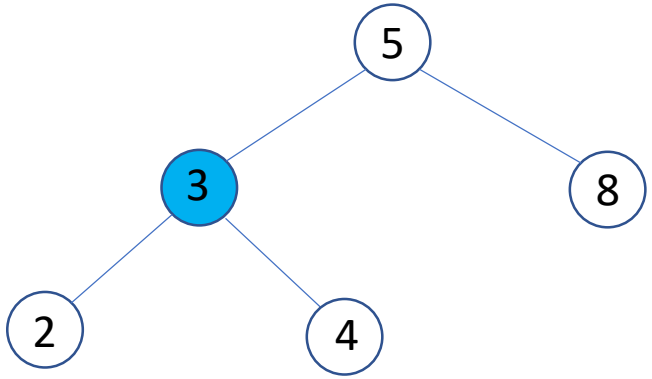
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

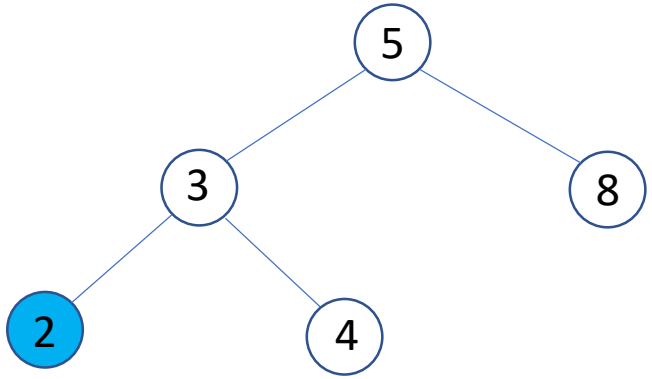
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

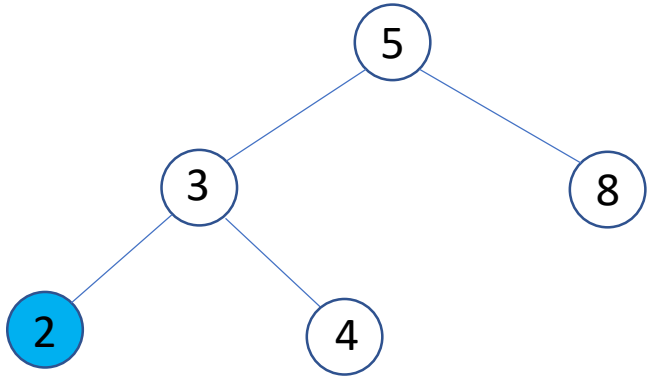
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

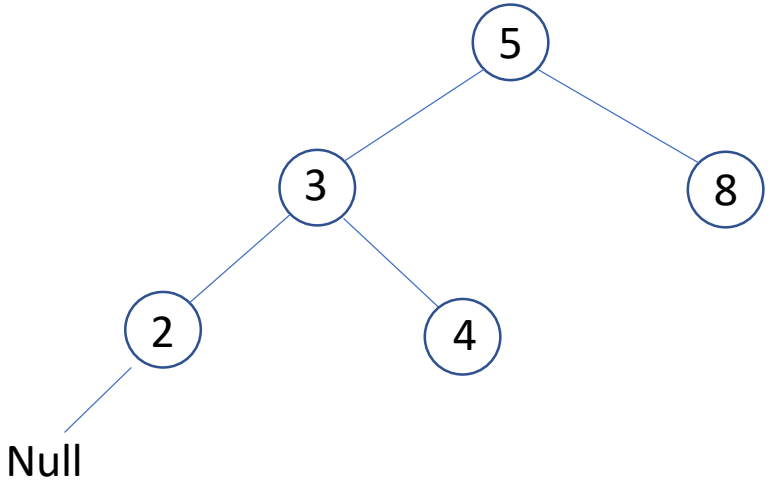
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

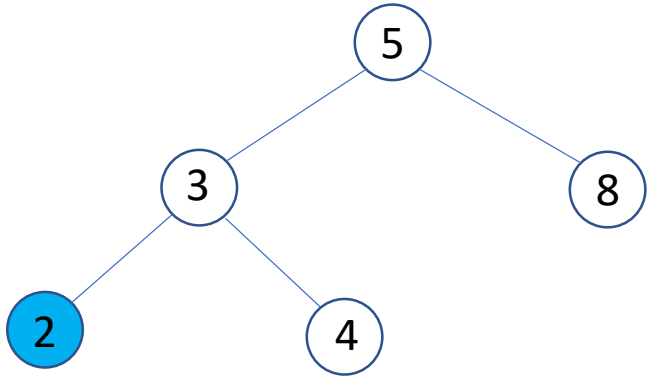
- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

# Caminhamento Central

- Caminhamento central e impressão da árvore:

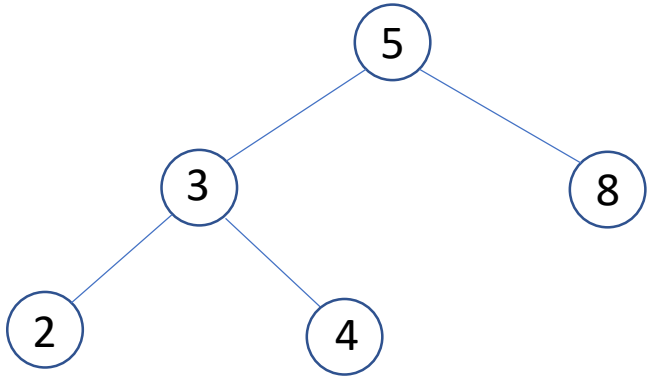



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2

# Caminhamento Central

- Caminhamento central e impressão da árvore:

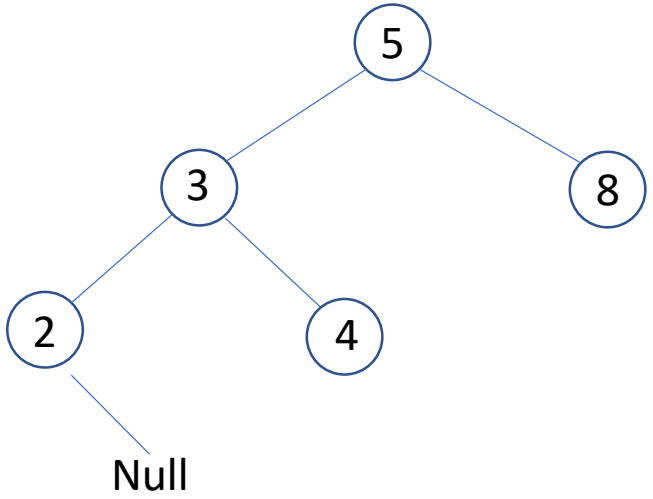



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2

# Caminhamento Central

- Caminhamento central e impressão da árvore:

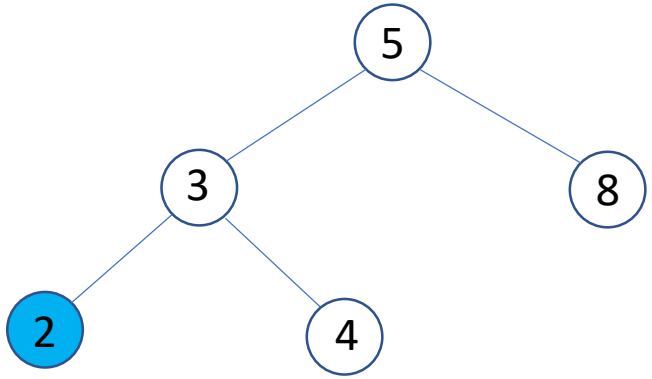



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2

# Caminhamento Central

- Caminhamento central e impressão da árvore:

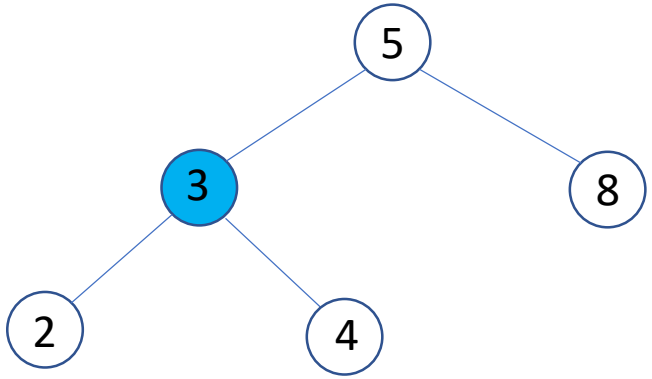



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2

# Caminhamento Central

- Caminhamento central e impressão da árvore:

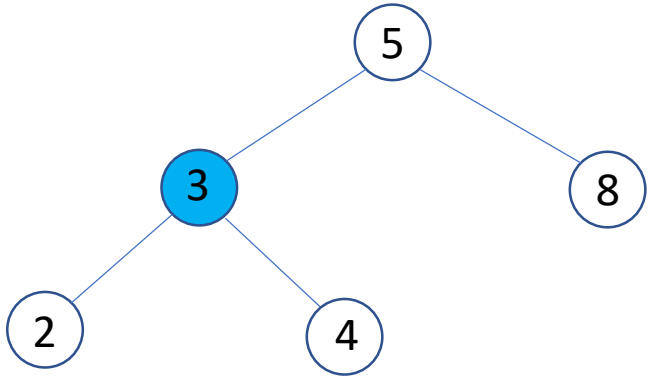



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3

# Caminhamento Central

- Caminhamento central e impressão da árvore:

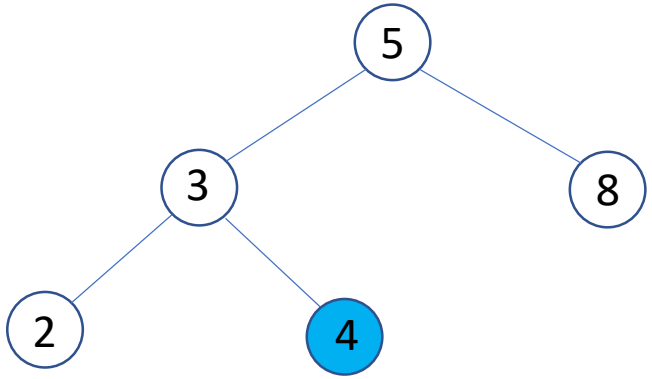



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3

# Caminhamento Central

- Caminhamento central e impressão da árvore:

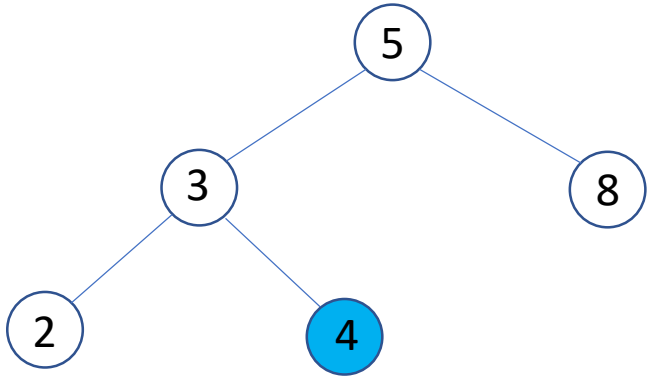



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3

# Caminhamento Central

- Caminhamento central e impressão da árvore:

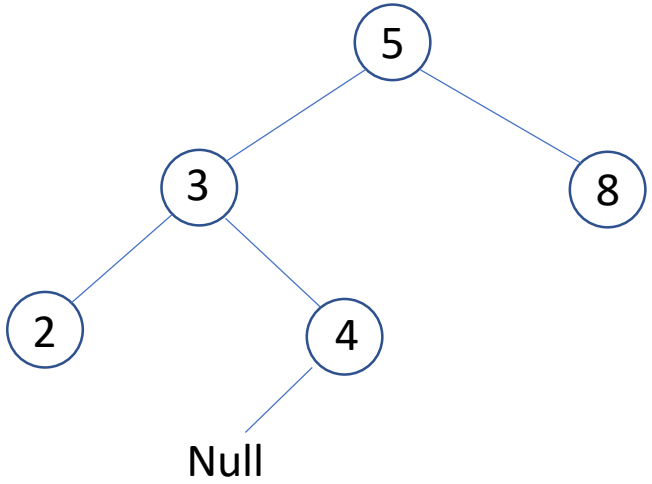



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3

# Caminhamento Central

- Caminhamento central e impressão da árvore:

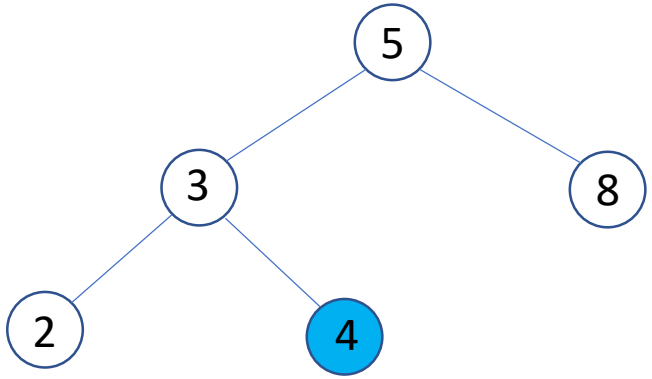



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3

# Caminhamento Central

- Caminhamento central e impressão da árvore:

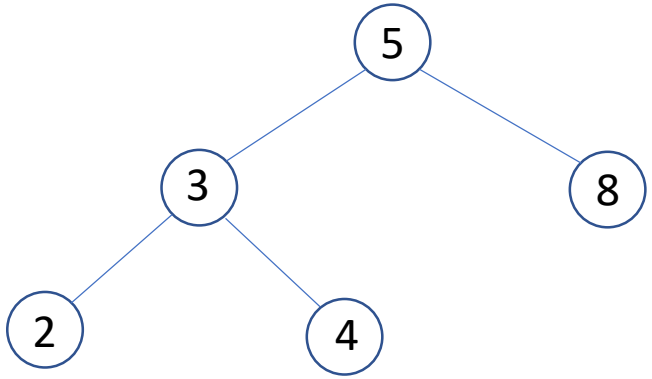



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4

# Caminhamento Central

- Caminhamento central e impressão da árvore:

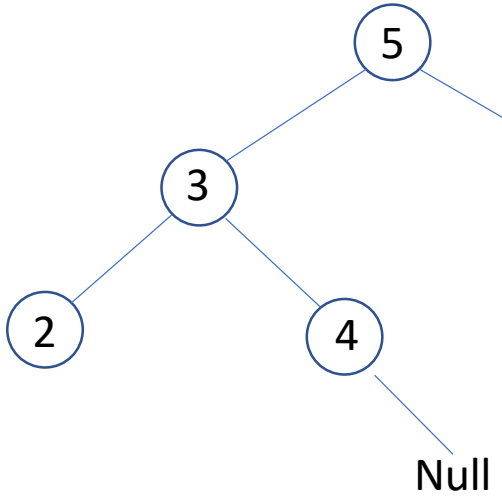



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4

# Caminhamento Central

- Caminhamento central e impressão da árvore:

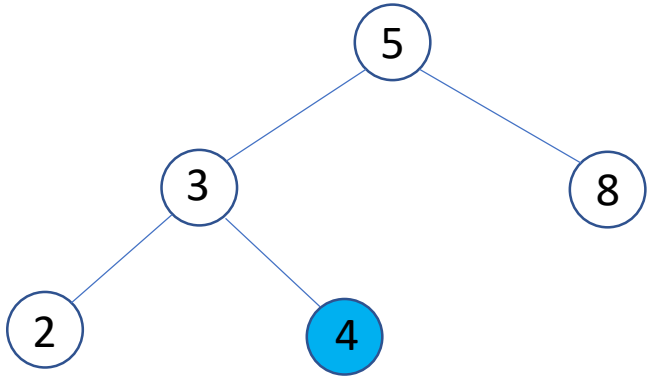



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4

# Caminhamento Central

- Caminhamento central e impressão da árvore:

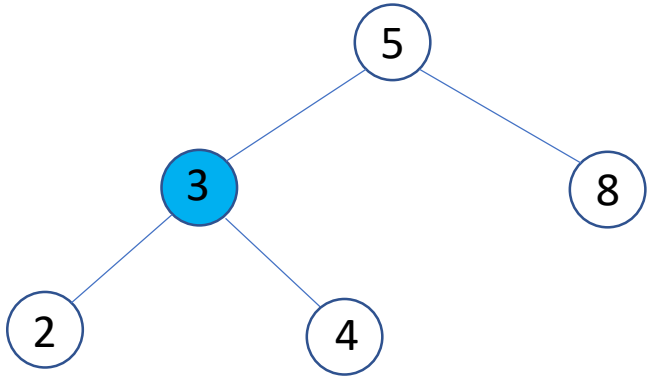



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4

# Caminhamento Central

- Caminhamento central e impressão da árvore:

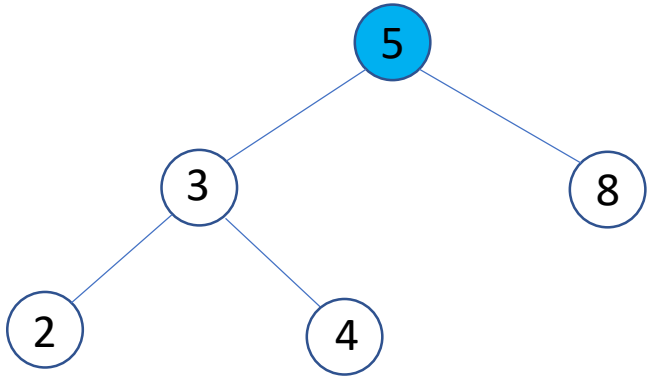



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4

# Caminhamento Central

- Caminhamento central e impressão da árvore:

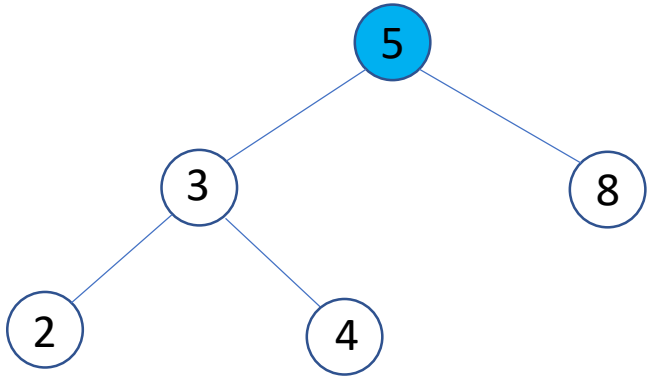



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5

# Caminhamento Central

- Caminhamento central e impressão da árvore:

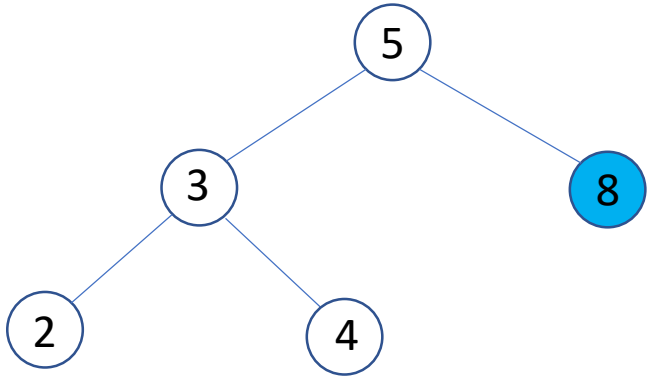



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5

# Caminhamento Central

- Caminhamento central e impressão da árvore:

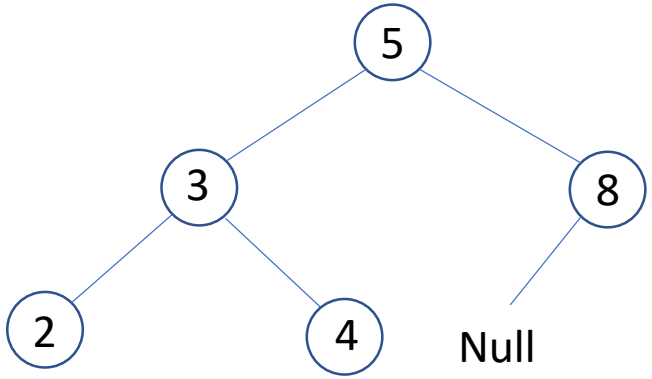



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5

# Caminhamento Central

- Caminhamento central e impressão da árvore:

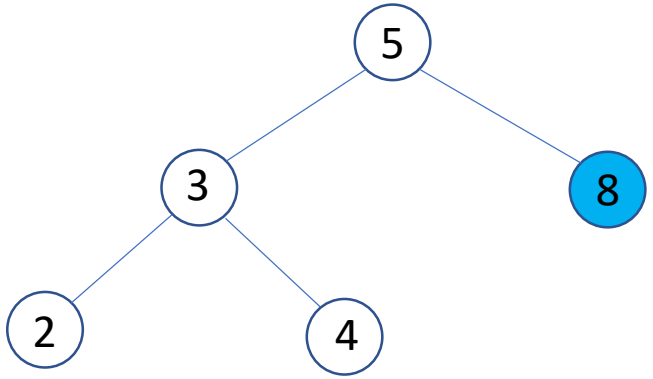



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5

# Caminhamento Central

- Caminhamento central e impressão da árvore:

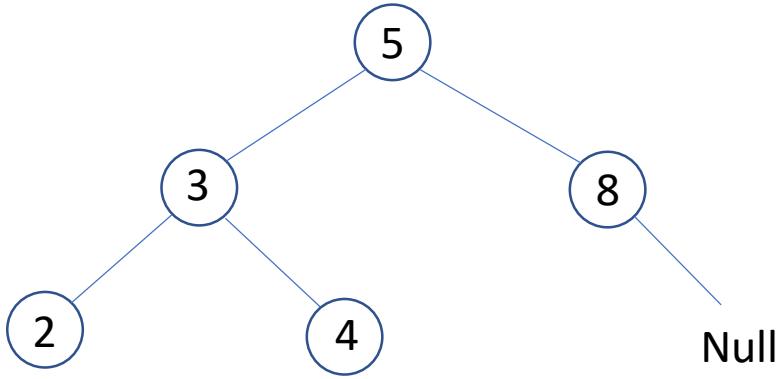



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5 8

# Caminhamento Central

- Caminhamento central e impressão da árvore:

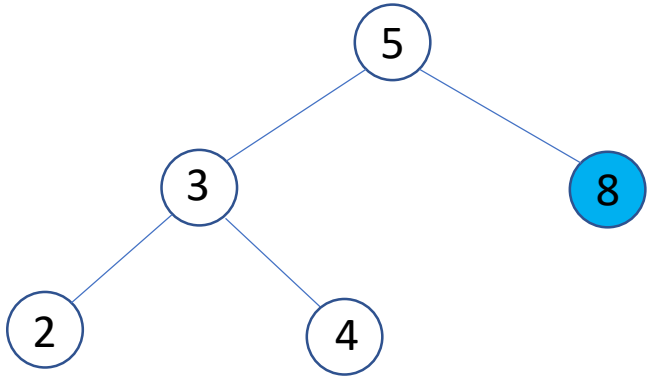



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5 8

# Caminhamento Central

- Caminhamento central e impressão da árvore:

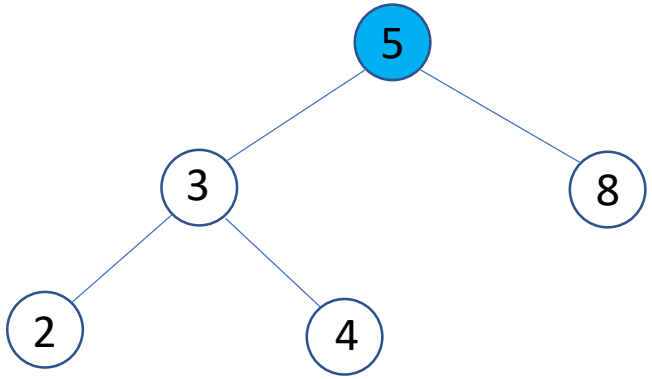



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5 8

# Caminhamento Central

- Caminhamento central e impressão da árvore:

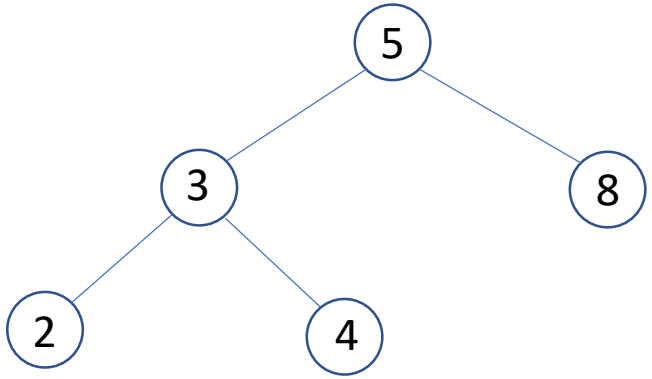



```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

2 3 4 5 8

# Caminhamento Central

- Caminhamento central e impressão da árvore:




```
private void central(No p) {
    if (p != null) {
        central(p.esq);
        System.out.println(p.reg.toString());
        central(p.dir);
    }
}
```

Uma característica importante do caminhamento central é que os nós são visitados de **forma ordenada**.

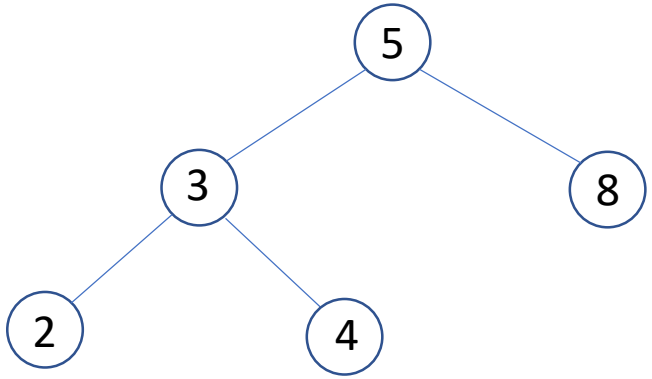
2 3 4 5 8

# Caminhamento Pré-ordem

- O caminhamento em **pré-ordem** lista o **nó raiz da sub árvore**, **o nó filho da sub árvore à esquerda** e lista o **nó filho à direita**. (em cada subárvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **raiz, nós filhos à esquerda** e **nós filhos à direita**.

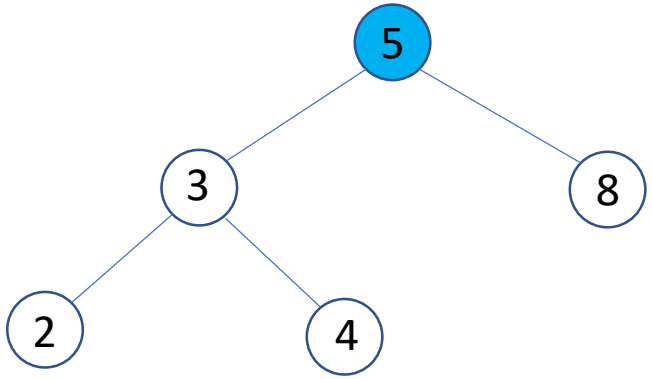
# Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:



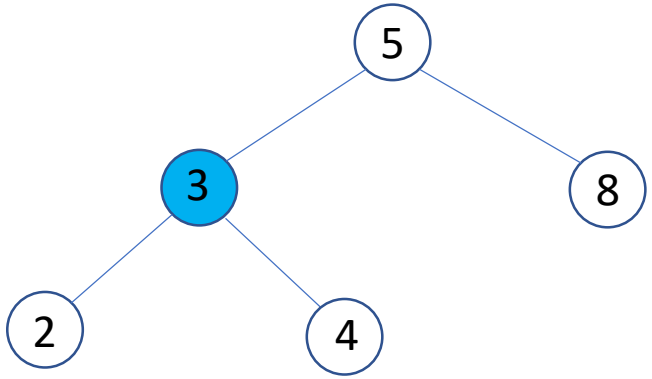
# Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:



# Caminhamento Pré-ordem

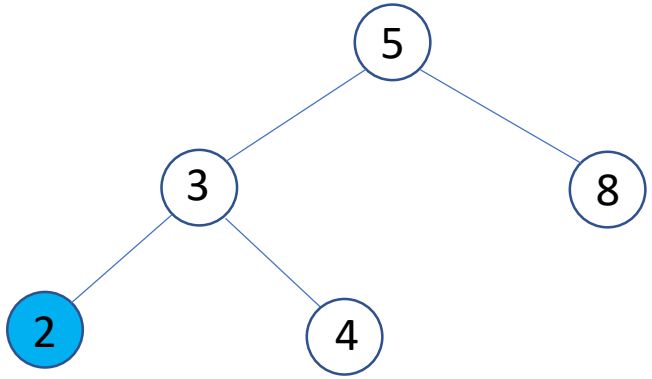
- Caminhamento Pré-ordem e impressão da árvore:




5 3

# Caminhamento Pré-ordem

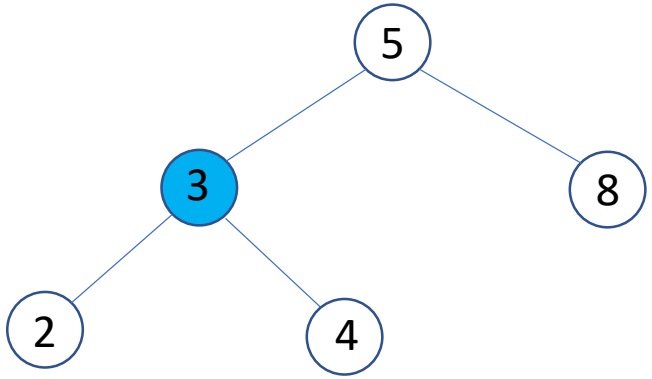
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2

# Caminhamento Pré-ordem

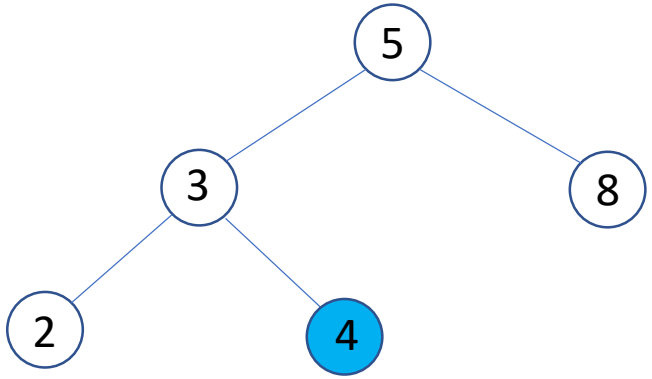
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2

# Caminhamento Pré-ordem

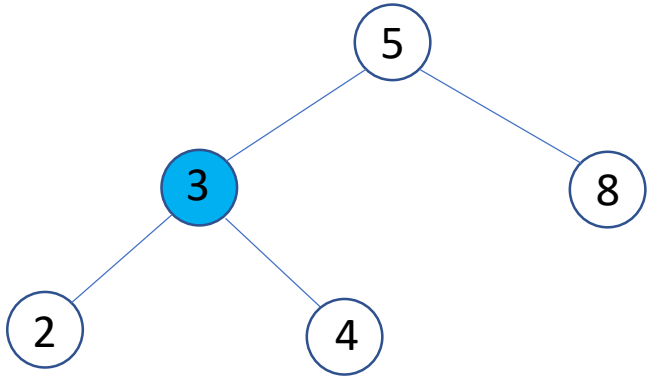
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2 4

# Caminhamento Pré-ordem

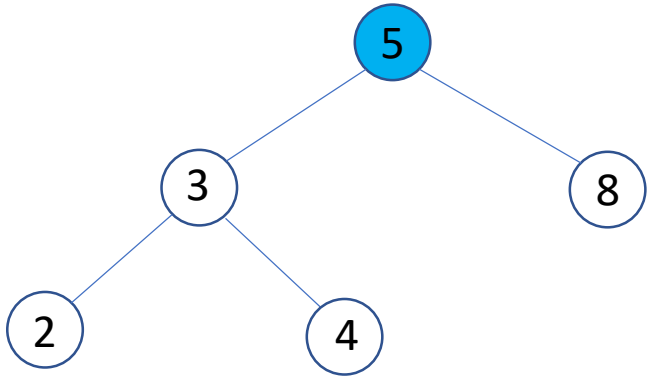
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2 4

# Caminhamento Pré-ordem

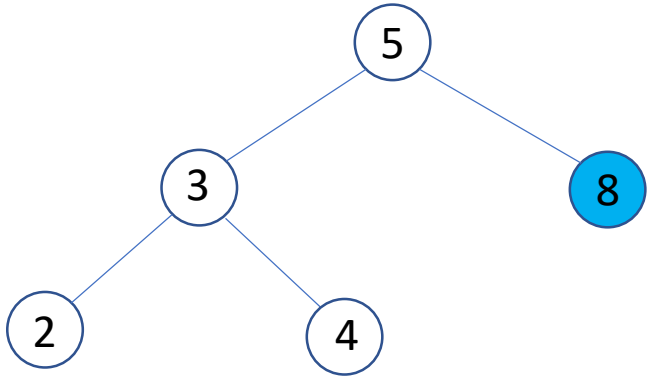
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2 4

# Caminhamento Pré-ordem

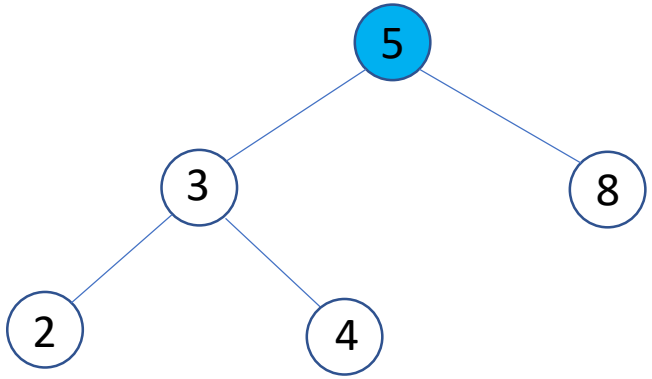
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2 4 8

# Caminhamento Pré-ordem

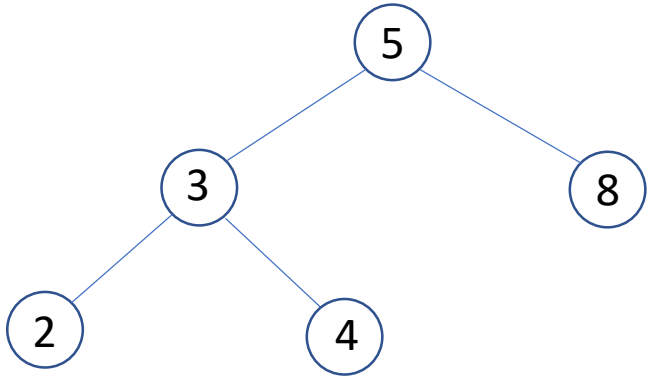
- Caminhamento Pré-ordem e impressão da árvore:




5 3 2 4 8

# Caminhamento Pré-ordem

- Caminhamento Pré-ordem e impressão da árvore:




Observe que o nó raiz de cada sub árvore é listado primeiro.

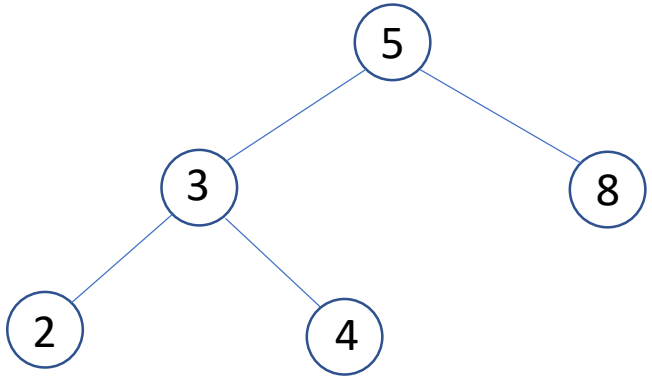
5 3 2 4 8

# Caminhamento Pós-ordem

- O caminhamento em **pós-ordem** lista os **nós filhos à esquerda** da sub árvore, os **nós filhos da sub árvore à direita** e lista o **nó raiz** da sub árvore. (em cada sub árvore lista os nós sempre da esquerda para a direita). O procedimento é aplicado recursivamente a toda árvore.
  - Em resumo: os nós de cada sub árvore são listados nesta ordem: **nós filhos à esquerda**, **nós filhos à direita** e **raiz**.

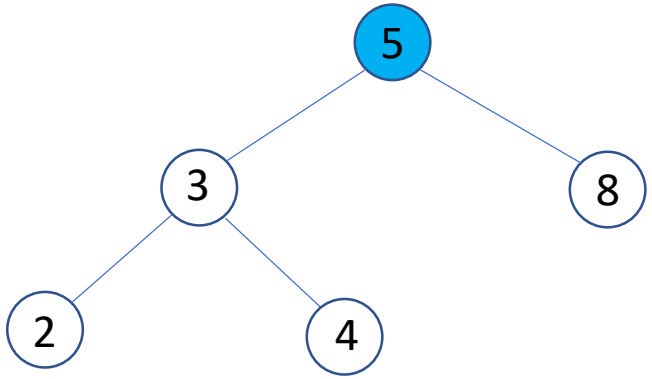
# Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



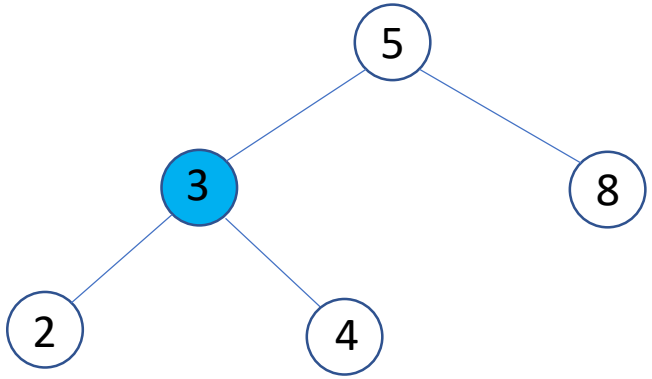
# Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



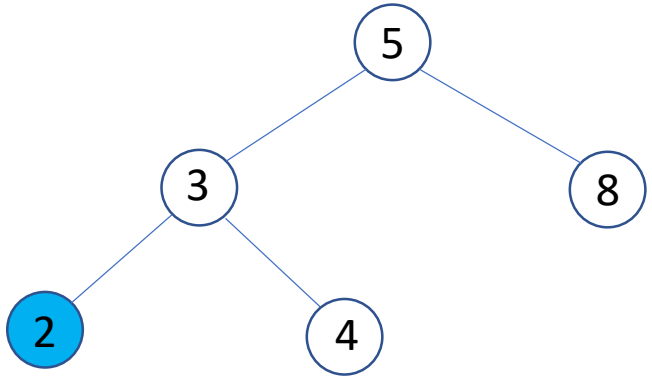
# Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



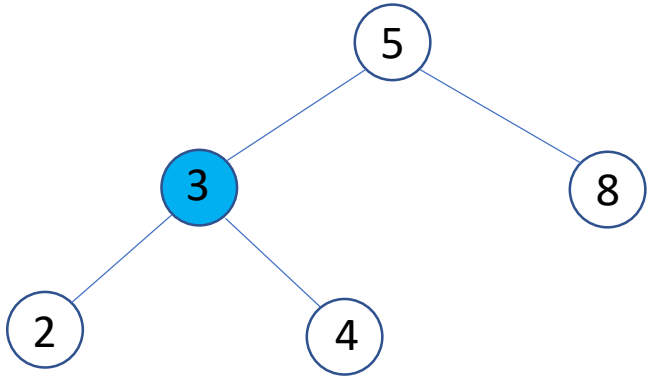
# Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



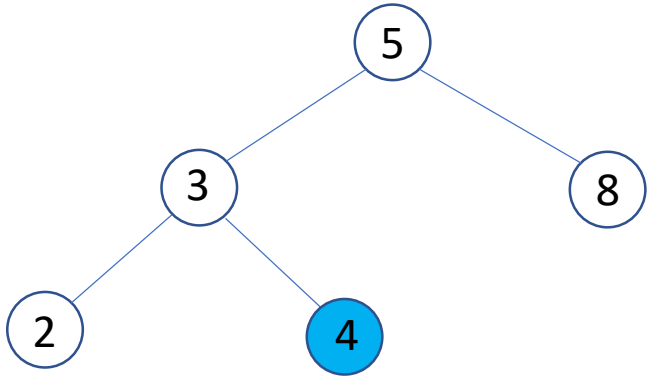
# Caminhamento Pós-ordem

- Caminhamento Pós-ordem e impressão da árvore:



# Caminhamento Pós-ordem

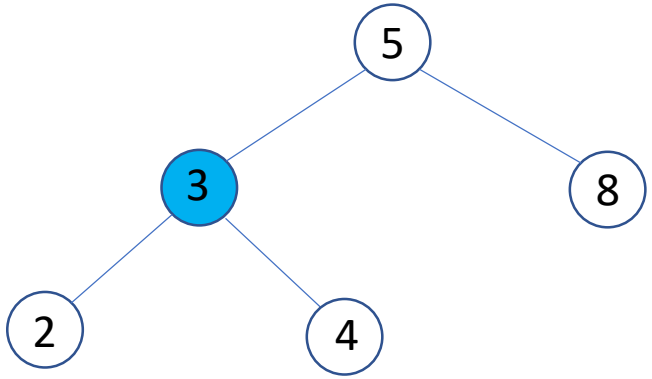
- Caminhamento Pós-ordem e impressão da árvore:




2 4

# Caminhamento Pós-ordem

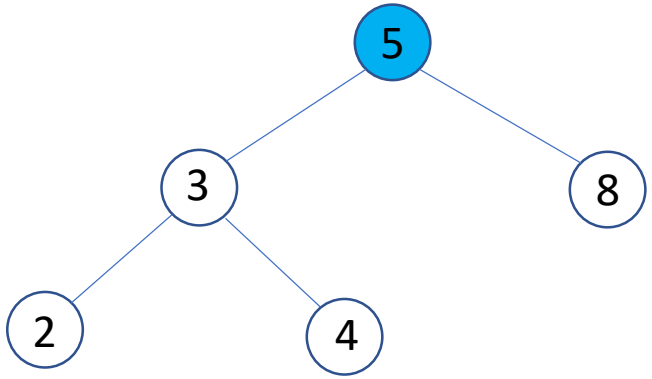
- Caminhamento Pós-ordem e impressão da árvore:




2 4 3

# Caminhamento Pós-ordem

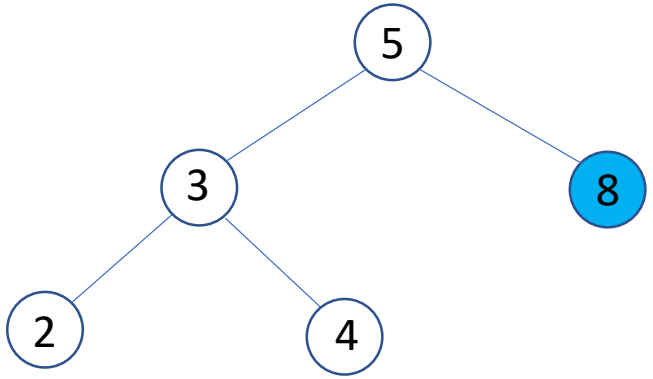
- Caminhamento Pós-ordem e impressão da árvore:




2 4 3

# Caminhamento Pós-ordem

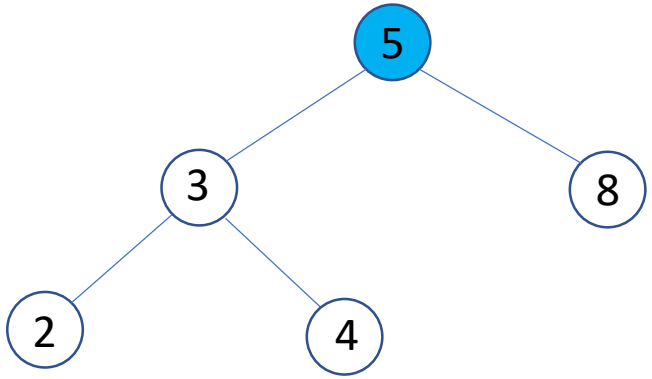
- Caminhamento Pós-ordem e impressão da árvore:




2 4 3 8

# Caminhamento Pós-ordem

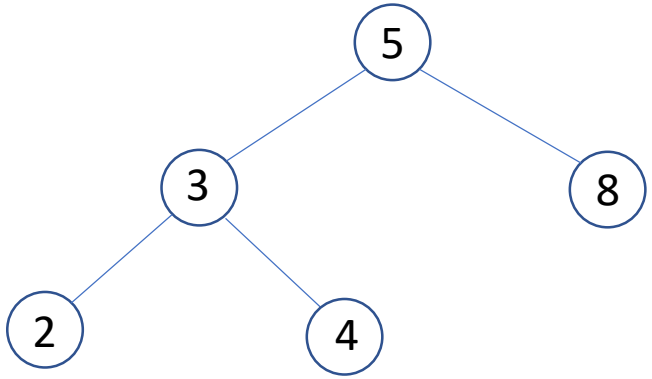
- Caminhamento Pós-ordem e impressão da árvore:




2 4 3 8 5

# Caminhamento Pós-ordem

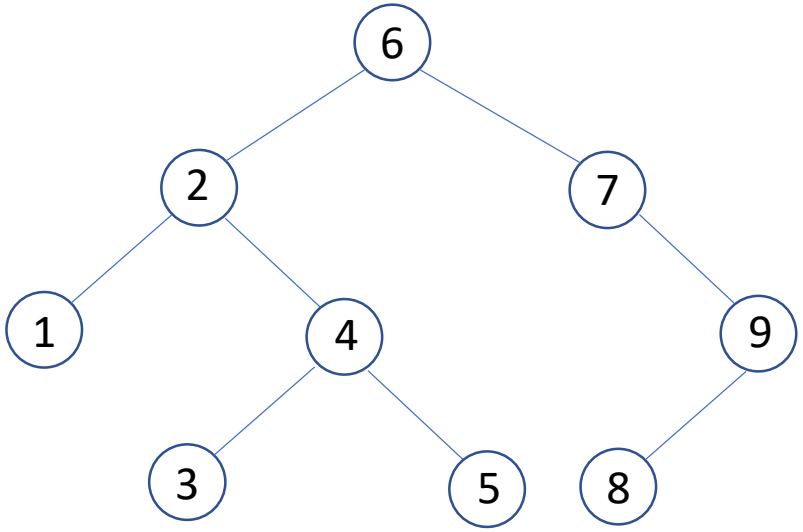
- Caminhamento Pós-ordem e impressão da árvore:




Observe que o nó raiz de cada sub árvore é listado por último.

2 4 3 8 5

# Resumo sobre caminhamento




Pré-ordem: 6 – 2 – 1 – 4 – 3 – 5 – 7 – 9 – 8 (para cada sub árvore, a raiz é listada primeiro: **pré**)
Central: 1 – 2 – 3 – 4 – 5 – 6 – 7 – 8 – 9 (para cada sub árvore, a raiz é listada no centro: **central**)
Pós-Ordem: 1 – 3 – 5 – 4 – 2 – 8 – 9 – 7 – 6 (para cada sub árvore, a raiz é listada por último: **pós**)

# Análise

- O número de comparações em uma pesquisa com sucesso:
  - melhor caso: $C(n) = (1)$
  - pior caso: $C(n) = (n)$ ← ???
  - caso médio: $C(n) = (Log\ n)$.

# Análise

- O número de comparações em uma pesquisa com sucesso:
  - melhor caso: $C(n) = (1)$
  - pior caso: $C(n) = (n)$ ← ???
  - caso médio: $C(n) = (Log\ n)$.

- O tempo de execução dos algoritmos para árvores binárias de pesquisa depende muito do **formato das árvores**.

# Análise

- Para obter o pior caso basta que as chaves sejam inseridas em ordem crescente ou decrescente.
  - Árvore se torna uma lista linear.
  - tempo médio de comparações: $(n + 1)/2$.

# Exercícios

- Implemente a classe ArvoreBinaria, apresentada em sala, e teste você deve criar e inserir, pesquisar e remover uma série de elementos de uma árvore de pesquisa binária.
- Você deve acrescentar métodos à classe Arvore que permitam imprimir todas as informações dos elementos ou apenas as suas chaves durante a realização de caminhamentos (central, pré-ordem e pós-ordem).

# Exercícios

- Modifique sua implementação de modo a contabilizar para sua classe árvore o número de comparações realizadas durante as operações de inserção, de pesquisa e de remoção.

# Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

# Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).